

1.6. Modeling

It should go without saying that the focus of UML is modeling. However, what that means, exactly, can be an open-ended question. Modeling is a means to capture ideas, relationships, decisions, and requirements in a well-defined notation that can be applied to many different domains. Modeling not only means different things to different people, but also it can use different pieces of UML depending on what you are trying to convey.

In general a UML model is made up of one or more *diagrams*. A diagram graphically represents things, and the relationships between these things. These things can be representations of real-world objects, pure software constructs, or a description of the behavior of some other object. It is common for an individual thing to show up on multiple diagrams; each diagram represents a particular interest, or *view*, of the thing being modeled.

1.6.1. Diagrams

UML 2.0 divides diagrams into two categories: *structural diagrams* and *behavioral diagrams*. Structural diagrams are used to capture the physical organization of the things in your system, i.e., how one object relates to another. There are several structural diagrams in UML 2.0:

Class diagrams

Class diagrams use classes and interfaces to capture details about the entities that make up your system and the static relationships between them. Class diagrams are one of the most commonly used UML diagrams, and they vary in detail from fully fleshed-out and able to generate source code to quick sketches on whiteboards and napkins. Class diagrams are discussed in [Chapter 2](#).

Component diagrams

Component diagrams show the organization and dependencies involved in the implementation of a system. They can group smaller elements, such as classes, into larger, deployable pieces. How much detail you use in component diagrams varies depending on what you are trying to show. Some people simply show the final, deployable version of a system, and others show what functionality is provided by a particular component and how it realizes its functionality internally. Component diagrams are discussed in [Chapter 5](#).

Composite structure diagrams

Composite structure diagrams are new to UML 2.0. As systems become more complex, the relationships between elements grow in complexity as well. Conceptually, composite structure diagrams link class diagrams and component diagrams; they don't emphasize the design detail that class diagrams do or the implementation detail that composite structures do. Instead, composite structures show how elements in the system combine to realize complex patterns. Composite structures are discussed in [Chapter 4](#).

Deployment diagrams

Deployment diagrams show how your system is actually executed and assigned to various pieces of hardware. You typically use deployment diagrams to show how components are configured at runtime. Deployment diagrams are discussed in [Chapter 6](#).

Package diagrams

Package diagrams are really special types of class diagrams. They use the same notation but their focus is on how classes and interfaces are grouped together. Package diagrams are discussed in [Chapter 3](#).

Object diagrams

Object diagrams use the same syntax as class diagrams and show how actual instances of classes are related at a specific instance of time. You use object diagrams to show snapshots of the relationships in your system at runtime. Object diagrams are discussed as part of class diagrams in [Chapter 2](#).

Behavioral diagrams focus on the behavior of elements in a system. For example, you can use behavioral diagrams to capture requirements, operations, and internal state changes for elements. The behavioral diagrams are:

Activity diagrams

Activity diagrams capture the flow from one behavior or *activity*, to the next. They are similar in concept to a classic flowchart, but are much more expressive. Activity diagrams are discussed in [Chapter 9](#).

Communication diagrams

Communication diagrams are a type of interaction diagram that focuses on the elements involved in a particular behavior and what messages they pass back and forth. Communication diagrams emphasize the objects involved more than the order and nature of the messages exchanged. Communication diagrams are discussed as part of interaction diagrams in [Chapter 10](#).

Interaction overview diagrams

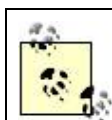
Interaction overview diagrams are simplified versions of activity diagrams. Instead of emphasizing the activity at each step, interaction overview diagrams emphasize which element or elements are involved in performing that activity. The UML specification describes interaction diagrams as emphasizing who has the *focus of control* throughout the execution of a system. Interaction overview diagrams are discussed as part of interaction diagrams in [Chapter 10](#).

Sequence diagrams

Sequence diagrams are a type of interaction diagram that emphasize the type and order of messages passed between elements during execution. Sequence diagrams are

1.6.4. Classifiers and Adornments

The basic modeling element in UML is the *classifier*. A classifier represents a group of things with common properties. Remember, at the level of classifier, we are discussing UML itself, not a particular system. So, when we say a *class* is a classifier, we mean that *classes* are things that have common properties: methods, attributes, exceptions, visibility, etc. A specific class, such as *Automobile*, isn't a UML classifier; it's an instance of a classifier, or a class.



For the truly self-abusing, this is a glimpse into the UML *meta-model*. The full metamodel is quite complex and begins with the UML infrastructure specification.

A classifier's generic notation is a rectangle that can be divided into *compartments* to show classifier-specific information, such as operations, attributes, or state activities. However, many UML classifiers such as states, activities, objects, etc., have custom notations to help distinguish them visually.

A classifier can have several types of extra information attached to it via a UML mechanism called *adornments*. For example, classifiers can have restrictions placed on the values a feature of the classifier can take. In general, constraints are written near the classifier or in an attached note. See the specific diagram types for details on what notation to use for a constraint when writing it near the classifier.

Another type of adornment is a *stereotype*. Just as you would expect, a stereotype is intended to give the reader a general idea of what a particular classifier represents. Stereotypes are usually associated with implementation concepts, such as «*Transactional*» or «*singleton*», though that isn't required by the UML specification.

[◀ PREV](#)[NEXT ▶](#)

Chapter 2. Class Diagrams

Class diagrams are one of the most fundamental diagram types in UML. They are used to capture the static relationships of your software; in other words, how things are put together.

When writing software you are constantly making design decisions: what classes hold references to other classes, which class "owns" some other class, and so on. Class diagrams provide a way to capture this "physical" structure of a system.

[◀ PREV](#)[NEXT ▶](#)

2.1. Classes

A class represents a group of things that have common state and behavior. You can think of a class as a blueprint for an object in an object-oriented system. In UML speak, a class is a kind of *classifier*. For example, Volkswagen, Toyota, and Ford are all cars, so you can represent them using a class named `Car`. Each specific type of car is an *instance* of that class, or an *object*. A class may represent a tangible and concrete concept, such as an invoice; it may be abstract, such as a document or a vehicle (as opposed to an invoice, or a motorcycle greater than 1000 cc), or it may represent an intangible concept such as a high-risk investment strategy.

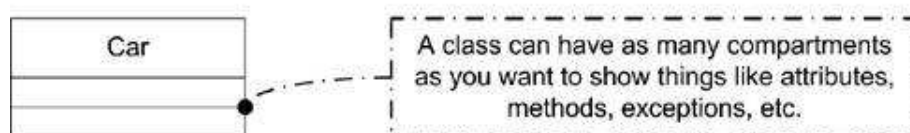
You represent a class with a rectangular box divided into *compartments*. A compartment is simply an area in the rectangle to write information. The first compartment holds the name of the class, the second holds attributes (see "[Attributes](#)"), and the third is used for operations (see "[Operations](#)"). You can hide any compartment of the class if that increases the readability of your diagram. When reading a diagram, you can make no assumptions about a missing compartment; it doesn't mean it is empty. You may add compartments to a class to show additional information, such as exceptions or events, though this is outside of the typical notation.

UML suggests that the class name:

- Start with a capital letter
- Be centered in the top compartment
- Be written in a boldface font
- Be written in italics if the class is *abstract* (see "[Abstract Classes](#)")

[Figure 2-1](#) shows a simple class.

Figure 2-1. Simple class representation

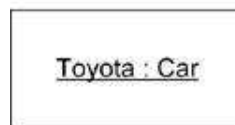


2.1.1. Objects

An object is an *instance* of a class. For example, you may have several instances of a class named `Car`: one two-door red car, one four-door blue car, and one hatchback green car. Each

instance of `Car` is an object and may be given its own name, though it is common to see unnamed, or anonymous, objects on object diagrams. Typically you show the name of the object followed by a colon followed by its type (i.e., class). You show that this is an instance of a class by underlining the name and type. [Figure 2-2](#) shows an instance of a class `Car` named `Toyota`. Note that in this figure, we have hidden the empty compartments.

Figure 2-2. An instance of Car



2.2. Attributes

Details of a class (the color of a car, the number of sides in a shape, etc.) are represented as *attributes*. Attributes can be simple primitive types (integers, floating-point numbers, etc.) or relationships to other, complex objects (see "[Relationships](#)").

An attribute can be shown using two different notations: inlined or relationships between classes. In addition, notation is available to show such things as multiplicity, uniqueness, and ordering. This section introduces both notations, and then describes the details of the attribute specification.

2.2.1. Inlined Attributes

You can list a class's attributes right in rectangle notation; these are typically called *inlined attributes*. There is no semantic difference between inlined attributes and attributes by relationship; it's simply a matter of how much detail you want to present (or, in the case of primitives like integers, how much detail you *can* present).

To represent an attribute within the body of a class, place the attribute in the second compartment of the class. UML refers to inlined attributes as *attribute notation*. Inlined attributes use the following notation:

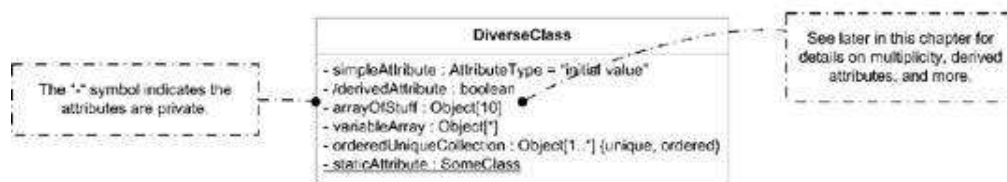
```
visibility / name : type multiplicity = default
{property strings and constraints }
```

```
visibility ::= {+|-|#|~}
```

```
multiplicity ::= [lower..upper]
```

[Figure 2-3](#) lists several attributes, demonstrating various aspects of attribute notation.

Figure 2-3. Example attributes



The syntax elements are:

visibility

Indicates the visibility of the attribute. Use the following symbols: +, -, #, or ~ for `public`, `private`, `protected`, or `package`, respectively (see "[Visibility](#)" in [Chapter 3](#)).

/

Indicates the attribute is *derived*. A derived attribute is simply one that can be computed from other attributes of the class. See "[Derived Attributes](#)."

name

Is a noun or short phrase naming the attribute. Typically the first letter is lowercase, and the first letter of each subsequent word is capitalized.

type

Is the type of the attribute as another classifier, typically a class, interface, or built-in type like `int`.

multiplicity

Specifies how many instances of the attribute's type are referenced by this attribute. Can be absent (meaning multiplicity of 1), a single integer, or a range of values specified between square brackets separated by "..". Use * as the upperbound to represent the upper limit or * on its own to mean zero or more. See "[Multiplicity](#)."

default

Is the default value of the attribute.

property strings

Is a collection of properties, or tags, that can be attached to attributes. These are typically context-specific and denote such things as ordering or uniqueness. They are surrounded by {} and separated by commas. See "Properties."

constraints

Are one or more restrictions placed on an attribute. They may be natural language or use a formal grammar such as the OCL. See "[Constraints](#)."

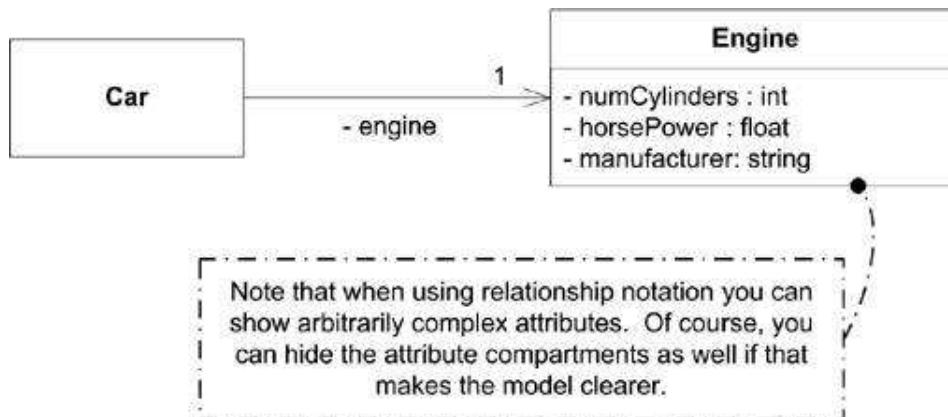
2.2.2. Attributes by Relationship

You may also represent attributes using the relationship notation. The relationship notation results in a larger class diagram, but it can provide greater detail for complex attribute types. The relationship notation also conveys exactly how the attribute is contained within a class (see "[Relationships](#)" for information on the types of relationships). For example, if you are modeling a `Car`, you can show that a car *contains* an `Engine` much more clearly using relationships than you can just by listing an attribute in the `Car`'s rectangle. However, showing

theCar's name by relationship is probably overkill because it is likely just a string.

To represent an attribute using relationships you use one of the association relationships between the class containing the attribute and the class that represents the attribute, as shown in [Figure 2-4](#), which shows that the relationship between a car and its engine has a multiplicity of 1; a car has one engine.

Figure 2-4. Attribute using relationship notation

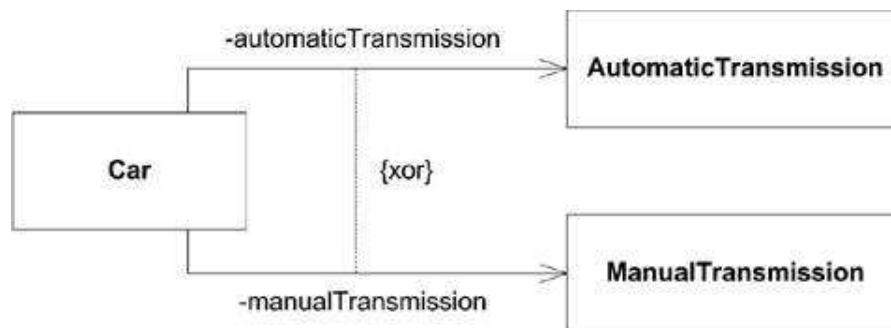


Yes, yes, as my editor pointed out, some cars like the Toyota Prius have two engines. Work with me here.

Relationship notation supports the same syntax as inlined notation, though the layout is slightly different. The attribute's visibility and name are placed near the relationship line. Don't use square brackets for multiplicity, but do place the multiplicity specification near the attribute's classifier.

Like multiplicity, you can place constraints on attributes (see "[Constraints](#)"). In relationship notation, you write constraints near the attribute's classifier along the relationship line. UML allows relationship notation to also express constraints between attributes, as shown in [Figure 2-5](#).

Figure 2-5. Relationship notation using constraints

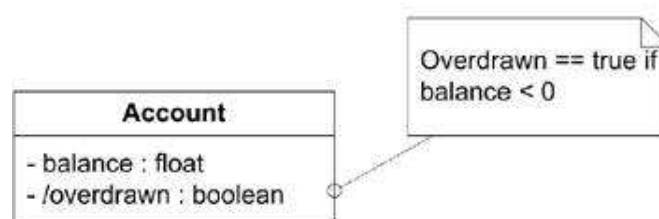


In [Figure 2-5](#), the standard UML constraint `xor` shows that only `automaticTransmission` or `manualTransmission` can be set at any given time (exclusive or). You need to express this constraint in a note if the inlined attribute notation was used.

2.2.3. Derived Attributes

The derived notation, which is the leading forward slash (/), can be used as an indicator to the implementer that the attribute may not be strictly necessary. For example, let's say you modeled a bank account with a simple class named `Account`. This class stores the current balance as a floating-point number named `balance`. To keep track of whether this account is overdrawn, you add a `boolean` named `overdrawn`. Whether the account is overdrawn is really based on whether the balance is positive, not the boolean you added. You can indicate this to the developer by showing that `overdrawn` is a derived attribute, with its state based on `balance`. [Figure 2-6](#) shows how `balance` and `overdrawn` can be represented using a note to convey the relationship.

Figure 2-6. Derived attribute



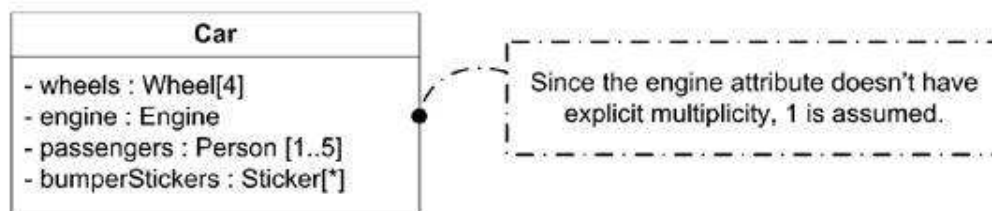
The UML specification notes that a derived attribute is typically `readOnly`, meaning a user may not modify the value. However, if a user is permitted to modify the value, the class is expected to update the source of the derived information appropriately.

2.2.4. Attribute Multiplicity

The multiplicity of an attribute specifies how many instances of the attribute's type are created when the owning class is instantiated. For example, our `Car` class will likely have four wheels, so the multiplicity of the wheel attribute is 4. If no multiplicity is specified, 1 is implied. Multiplicity can be a single integer, a list of integers separated by commas, or a range of values. When specifying a range of values, an infinite upper bound can be

represented as an `*`; if no lower bound is specified, an `*` means zero or more. The multiplicity value is shown between square brackets as a single integer or as two integers separated by two dots (`..`). [Figure 2-7](#) shows the various ways to represent an attribute's multiplicity.

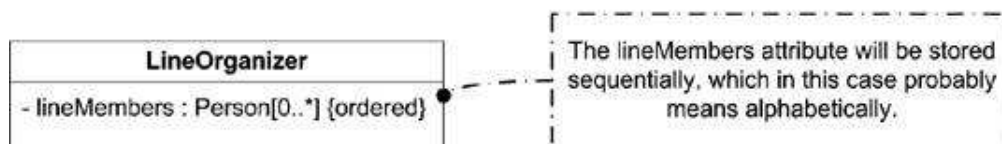
Figure 2-7. Multiplicity examples



2.2.4.1. Ordering

An attribute with a multiplicity greater than 1 can be specified to be *ordered*. If the attribute is ordered, the elements must be stored sequentially. For example, you can specify that a list of names be stored alphabetically by marking the list as ordered. Exactly what it means for attributes to be stored sequentially typically depends on the attribute type. By default, attributes are *not* ordered. To mark an attribute as ordered, specify the property `ordered` after the attribute in braces, as shown in [Figure 2-8](#).

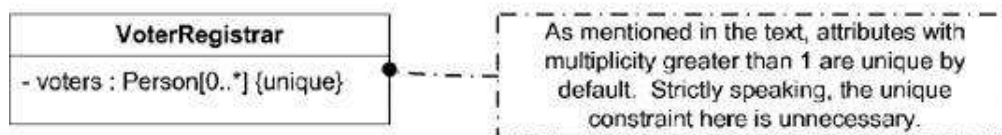
Figure 2-8. Ordered multiplicity



2.2.4.2. Uniqueness

In addition to being ordered, an attribute with multiplicity greater than 1 may be required to be *unique*. If an attribute is required to be unique, each element of this attribute must be unique. By default, attributes with multiplicity greater than 1 *are unique*, meaning there can be no duplicates in the elements this attribute holds. For example, if a class held a list of voters and each person was allowed to vote only once, each element in the list would be unique. To make an attribute unique, place the keyword `unique` after the attribute in braces, as shown in [Figure 2-9](#). To allow an attribute to hold duplicates of an object, simply use the property `notunique`.

Figure 2-9. Unique multiplicity



2.2.4.3. Collection types

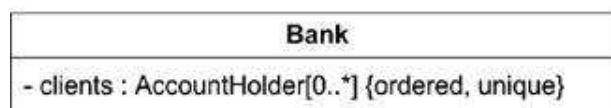
The UML specification specifies a set of mappings from the various ordered and uniqueness properties to UML collection types. [Table 2-1](#) shows the mappings from attribute properties to the UML collection type. Note that the collection types shown in [Table 2-1](#) are UML mappings and may not map directly to classes in a target language.

Table 2-1. Collection types for attributes

Order	Uniqueness	Associated collection type
False	False	Bag
True	True	OrderedSet
False	True	Set
True	False	Sequence

For example, to show that a bank's clients should be represented using an `OrderedSet`, you can model the `clients` attribute as shown in [Figure 2-10](#).

Figure 2-10. Example attribute stored in an OrderedSet



2.2.5. Attribute Properties

In addition to the properties associated with multiplicity, an attribute may have a number of properties set to convey additional information to the reader of the diagram. The common properties defined by UML are:

readOnly

Specifies that the attribute may not be modified once the initial value is set. This typically maps to a *constant* in a development language. UML doesn't specify when the initial value must be set, though if you specify the default value for an attribute it is considered the initial value and may not be changed.

union

Specifies that the attribute type is a union of the possible values for this attribute. Frequently this is used with the *derived* property to indicate that an attribute is a derived union of another set of attributes.

subsets <attribute-name>

Specifies that this attribute type is a subset of all the valid values for the given attribute. This isn't a common property, but if used, it is typically associated with subclasses of an attribute type.

redefines <attribute-name>

Specifies that this attribute acts as an alias for the given attribute. Though uncommon, this attribute can be used to show that a subclass has an attribute that is an alias for a superclass's attribute.

composite

Specifies that this attribute is part of a whole-part relationship with the classifier. See "[Relationships](#)" for more information on composition.

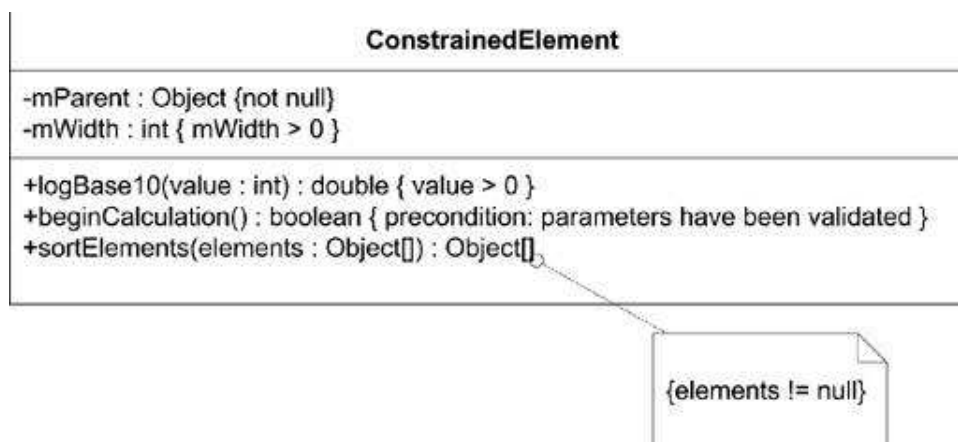
2.2.6. Constraints

Constraints represent restrictions placed on an element. They may be natural language or use a formal grammar such as the OCL; however, they must evaluate to a boolean expression. You typically show constraints between curly braces ({}) after the element they restrict, though they may be placed in a note and linked to the element using a dashed line.

You can name a constraint by specifying the name followed by a colon (:) before the boolean expression. This is frequently used to identify constraints on an operation (see "[Operation Constraints](#)").

[Figure 2-11](#) shows several constraints on attributes and operations.

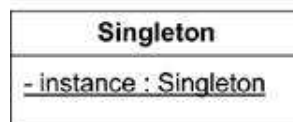
Figure 2-11. Examples of inlined and note constraints



2.2.7. Static Attributes

Static attributes are attributes of the class rather than of an instance of the class. For example, you can initialize constant values for a class and share them between all instances of the class. You represent static attributes by underlining their specification in both inlined and relationship-based presentations, as shown in [Figure 2-12](#).

Figure 2-12. Static attribute



2.3. Operations

Operations are features of classes that specify how to invoke a particular behavior. For example, a class may offer an operation to draw a rectangle on the screen or count the number of items selected in a list. UML makes a clear distinction between the specification of how to invoke a behavior (an operation) and the actual implementation of that behavior (a method). See "[Methods](#)" for more information.

You place operations in a separate compartment with the following syntax:

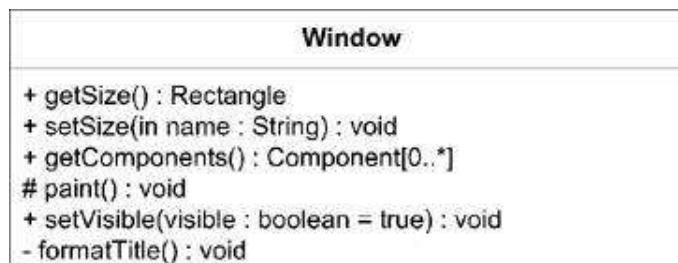
```
visibility name ( parameters ) : return-type { properties }
```

where parameters are written as:

```
direction parameter_name : type [ multiplicity ]  
= default_value { properties }
```

[Figure 2-13](#) shows several example operations on a class.

Figure 2-13. Example operations on a class



The syntax elements are:

visibility

Indicates the visibility of the operation. Use the following symbols: +, -, #, or ~ for public, private, protected, or package, respectively (see "[Visibility](#)" in [Chapter 3](#)).

name

Is a short phrase naming the operation. Operations are usually verb phrases representing actions the classifier should perform on behalf of the caller. The UML

specification recommends that the first letter of an operation be lowercase, with all of the following words starting with a capital letter and running together. See [Figure 2-13](#) for an example.

return-type

Is the type of information the operation will return, if any. If no information is returned from the operation (called a *subroutine* in some development languages), the return type should be *void*. If the operation does return a value (called a *function* in some development languages), you should show the type of the returned value, such as another classifier, a primitive type, or a collection. The UML specification states that the return type is optional. If it's left off, you can't assume anything about the return value of the operation, or even if one exists.

properties

Specifies constraints and properties associated with an operation. These are optional; if you don't use properties you don't show the curly braces. See "[Operation Constraints](#)" for more information.

Parameter syntax elements are:

direction

An optional part of the syntax that indicates how a parameter is used by an operation. It is one of *in*, *inout*, *out*, or *return.in* states that the parameter is passed to the operation by the caller. *inout* states that the parameter is passed by the caller and is then possibly modified by the operation and passed back out. *out* states that the parameter isn't set by the caller but is modified by the operation and is passed back out. *return* indicates that the value set by the caller is passed back out as a return value.

parameter_name

Is a noun or noun phrase naming the parameter. Typically the parameter name starts with a lowercase letter, with any subsequent words starting with a capital letter.

type

Is the type of the parameter. This is typically another class, interface, collection, or primitive type.

multiplicity

Specifies how many instances of the parameter's type are present. Can be absent (meaning multiplicity of 1), a single integer, a list of integers separated by commas, or a range of values specified between square brackets separated by "..". An infinite upper bound can be represented as an ***; if no lower bound specified, an *** means zero or more. See "[Multiplicity](#)."

default_value

Specifies the default value of this parameter. The default value is optional. If it isn't present, you don't show the equals sign. Note that UML doesn't specify if a parameter with a default value may be left out when invoking an operation (e.g., C++'s default parameter value implementation). The actual syntax for invoking an operation is language-dependent.

properties

Specifies any parameter-related properties and is specified between curly braces. These are typically defined within the context of a specific model, with a few exceptions: *ordered*, *readOnly*, and *unique*. See "[Multiplicity](#)" and "Properties" for more information. Properties are optional for parameters; if they aren't used, don't show the curly braces.

2.3.1. Operation Constraints

An operation may have several constraints associated with it that help define how the operation interacts with the rest of the system. Together, constraints on an operation establish a contract that an implementation of the operation must obey.

Constraints on an operation follow the usual constraint notation and are placed either immediately after the operation signature or in a note attached with a dashed line. See "[Constraints](#)" for more information.

2.3.1.1. Preconditions

Preconditions capture what the state of the system must be before an operation can be invoked. Practically speaking, you really can't express the state of the *entire* system. Instead, preconditions typically express the valid values for parameters, the state of the class owning the operation, or a few key attributes of the system.

The specification explicitly states that the operation doesn't need to check the preconditions in the body of the operation before executing; theoretically the operation will not even be invoked if the preconditions aren't met. In practice, few languages offer such protection. If someone took the time to express them, it is usually in your best interest to verify the preconditions are correct when implementing an operation.

As a developer, preconditions offer you one of the few chances to "cover your butt" and say exactly how you expect things to be when your implementation is invoked; use them.

[Figure 2-14](#) shows several examples of preconditions.

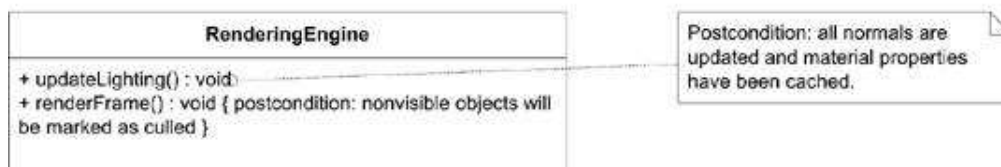
Figure 2-14. Preconditions for operations

2.3.1.2. Postconditions

Postconditions capture guarantees about the state of the system after an operation has executed. Like preconditions, postconditions typically express the state of one or more key attributes of the system, or some guarantee about the state of the class owning the operation.

[Figure 2-15](#) shows example postconditions linked to an operation.

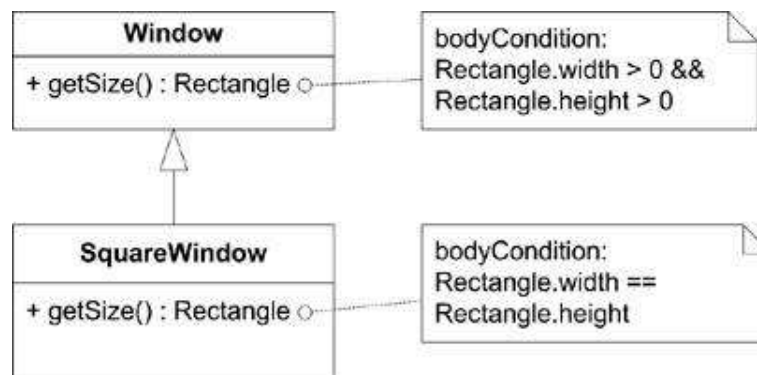
Figure 2-15. Postconditions for operations



2.3.1.3. Body conditions

An operation may have a `bodyCondition` that constrains the return value. The `bodyCondition` is separate from the postcondition because the `bodyCondition` may be replaced by methods of subclasses of the owning class. (See "[Generalization](#)" for more information on subclasses.) For example, a class named `Window` may specify a body condition for a method named `getSize( )` that requires the length and width of a window be nonzero. A subclass named `SquareWindow` may provide its own body condition stating that the width must equal the height. The `bodyCondition` is similar to the pre- and postconditions in that the constraint may be expressed in natural language or in OCL. See "[Constraints](#)" for more information. [Figure 2-16](#) shows an example of a `bodyCondition` on an operation.

Figure 2-16. Body conditions for operations

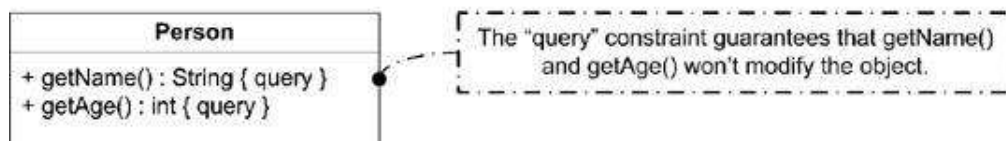


2.3.1.4. Query operations

An operation may be declared as a *query* operation if the implementation of the operation doesn't modify the owning class in any way. In practice, modelers often use the query property to indicate a method that doesn't change any meaningful attribute of an object. For example, there may be internal cache attributes that are updated as a result of a query. The important thing is that the state of the system, from an external perspective, isn't changed by the query method; there can be no side effects to calling the method.

You indicate a query method by placing the `query` constraint after the operation signature. For example, an operation named `getAge()` that simply returns an integer without changing any internal value of the owning class would be considered a query method. In C++, this typically maps to a `const` method. [Figure 2-17](#) shows several query methods on a class.

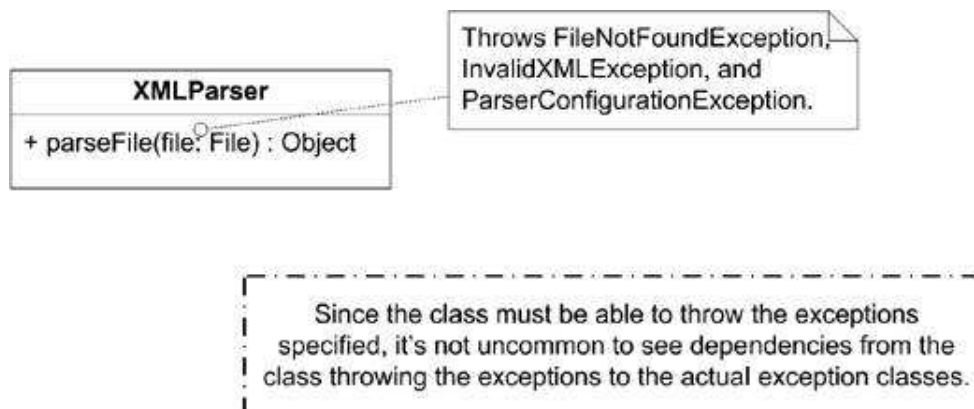
Figure 2-17. Example query operations



2.3.1.5. Exceptions

While they're technically not constraints, you may express exceptions thrown by an operation using similar notation. Exceptions are typically other classes (often stereotyped with the keyword `«exception»`, though this is simply by convention) that are thrown by an operation in the event of an error. You can list thrown exceptions in a note attached to an operation using a dashed line. [Figure 2-18](#) shows an example of an operation that throws several exceptions.

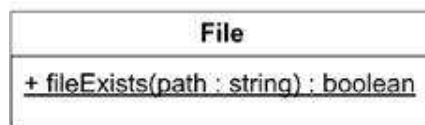
Figure 2-18. A method that throws several exceptions



2.3.2. Static Operations

Operations typically specify behavior for an *instance* of a class. However, UML allows for an operation to specify behavior for the *class itself*. These operations are called *static operations* and are invoked directly on the class, not on an instance. Static operations are frequently used as utility operations that don't need to use the attributes of the owning class. UML doesn't formally discuss the notation of these operations, but it is typical to see them represented with the same convention as static attributes. You indicate an operation is static by underlining the operation signature. [Figure 2-19](#) shows an example of a static operation.

Figure 2-19. A class with a static operation



[< PREV](#)[NEXT >](#)

2.4. Methods

A *method* is an implementation of an operation. Each class typically provides an implementation for its operations or inherits them from its superclass (see "[Generalization](#)"). If a class doesn't provide an implementation for an operation, and one isn't provided by its superclass, the operation is considered *abstract*. See "[Abstract Classes](#)" for more information. Because methods are implementations of operations, there is no notation for a method; simply show an operation on a class.

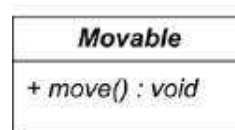
[< PREV](#)[NEXT >](#)

2.5. Abstract Classes

An *abstract class* is typically a class that provides an operation signature, but no implementation; however, you can have an abstract class that has no operations at all. An abstract class is useful for identifying common functionality across several types of objects. For example, you can have an abstract class named *Movable*. A *Movable* object has a current position and the ability to move somewhere else using an operation named `move()`. There can be several specializations of this abstract class: a *Car*, a *Grasshopper*, and a *Person*, each of which provides a different implementation of `move()`. Because the base class *Movable* doesn't have an implementation for `move()`, the class is said to be abstract.

You show a class is abstract by writing its name in italics. Show each abstract operation in italics as well. [Figure 2-20](#) is an example of the abstract class *Movable*.

Figure 2-20. Abstract class



An abstract class can't be instantiated; it must be subclassed and then a subclass which does provide the operation implementation can be instantiated. See "[Relationships](#)" for more information on subclasses.

2.6. Relationships

Classes in isolation would not provide much insight into how a system is designed. UML provides several ways of representing relationships between classes. Each of UML relationship represents a different type of connection between classes and has subtleties that aren't fully captured in the UML specification. When modeling in the real world, be sure that your intended viewers understand what you are conveying with your various relationships. We say this both as a warning to the modeler and as a slight disclaimer that the following explanations are our interpretation of the UML specification. For example, the debate over when to use aggregation versus composition is ongoing. To help determine which relationship is most appropriate, we offer a short phrase for each type that may help make the distinction. Again, the important thing is to be consistent within your model.

2.6.1. Dependency

The weakest relationship between classes is a *dependency* relationship. Dependency between classes means that one class uses, or has knowledge of, another class. It is typically a transient relationship, meaning a dependent class briefly interacts with the target class but typically doesn't retain a relationship with it for any real length of time.

Dependencies are typically read as "...uses a...". For example, if you have a class named `Window` that sends out a class named `WindowClosingEvent` when it is about to be closed, you would say "`Window` uses a `WindowClosingEvent`."

You show a dependency between classes using a dashed line with an arrow pointing from the dependent class to the class that is used. [Figure 2-21](#) shows a dependency between a class named `Window` and a class named `WindowClosingEvent`.

Figure 2-21. Window's dependency on WindowClosingEvent



You can likely assume from [Figure 2-21](#) that `Window` doesn't retain a relationship with a `WindowClosingEvent` for any real length of time. It simply uses them when needed and then forgets about them.

2.6.2. Association

Associations are stronger than dependencies and typically indicate that one class retains a

relationship to another class over an extended period of time. The lifelines of two objects linked by associations are probably not tied together (meaning one can be destroyed without necessarily destroying the other).

Associations are typically read as "...has a...". For example, if you have a class named `Window` that has a reference to the current mouse cursor, you would say "`Window` has a `Cursor`". Note that there is a fine line between "...has a..." and "...owns a..." (see "[Aggregation](#)" later in this section). In this case, `Window` doesn't own the `Cursor`; `Cursor` is shared between all applications in the system. However, `Window` has a reference to it so that the `Window` can hide it, change its shape, etc. You show an association using a solid line between the classes participating in the relationship. [Figure 2-22](#) shows an association between `Window` and `Cursor`.

Figure 2-22. Association showing Window "has a" Cursor



2.6.2.1. Navigability

Associations have explicit notation to express navigability. If you can navigate from one class to another, you show an arrow in the direction of the class you can navigate to. If you can navigate in both directions, it is common practice to not show any arrows at all, but as the UML specification points out, if you suppress all arrows you can't distinguish nonnavigable associations from two-way associations. However, it is extremely rare to use a nonnavigable association in the real world, so this is unlikely to be a problem.

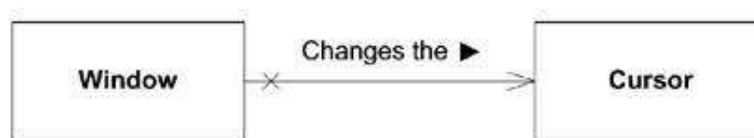
You can explicitly forbid navigation from one class to another by placing a small `x` on the association line at the end of the class you can't navigate to. [Figure 2-23](#) shows an association between a class named `Window` and a class named `Cursor`. Because you can't navigate from an instance of `Cursor` to an instance of `Window`, we explicitly show the navigability arrow and an `x` where appropriate.

Figure 2-23. Association between Window and Cursor showing you can't navigate from Cursor to Window

2.6.2.2. Naming an association

Associations may be adorned with several symbols to add information to your model. The simplest is a solid arrowhead showing the direction in which the viewer should read the association. It is common to include a short phrase along with the arrowhead to provide some context for the association. The phrase used with the association doesn't typically generate into any form of code representation; it is purely for modeling purposes. [Figure 2-24](#) shows the solid arrowhead on the `Window` to `Cursor` association.

Figure 2-24. Shows how to read the association between `Window` and `Cursor`



2.6.2.3. Multiplicity

Because associations typically represent lasting relationships, they are often used to indicate attributes of a class. As mentioned in the "Attributes by Relationship" section, you can express how many instances of a particular class are involved in a relationship. If you don't specify a value, a multiplicity of 1 is assumed. To show a different value, simply place the multiplicity specification near the owned class. See "[Attribute Multiplicity](#)" for the allowable multiplicity types. Note that when you use multiplicity with an association, you don't use square brackets around the values. [Figure 2-25](#) shows an association with explicit multiplicity.

Figure 2-25. A simple association showing four `Buttons` in a `Window`



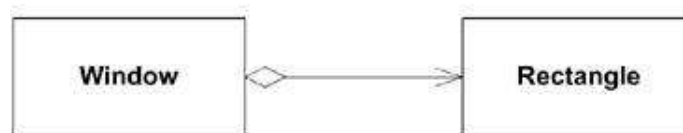
The properties related to multiplicity may be applied to associations as well. See "Properties" for the names and definitions of allowed properties.

2.6.3. Aggregation

Aggregation is a stronger version of association. Unlike association, aggregation typically implies ownership and may imply a relationship between lifelines. Aggregations are usually read as "...owns a...". For example, if you had a classed named `Window` that stored its position and size in a `Rectangle` class, you would say the "`Window` owns a `Rectangle`." The rectangle may be shared with other classes, but the `Window` has an intimate relationship with the `Rectangle`. This is subtly different from a basic association; it has a stronger connotation. However, it's not the strongest relationship you can have between classes. If the relationship is more of a whole part (class A "...is part of..." class B), you should look at *composition*.

You show an aggregation with a diamond shape next to the owning class and a solid line pointing to the owned class. [Figure 2-26](#) shows an example aggregation between a class named `Window` and a class named `Rectangle`.

Figure 2-26. Window "owns a" Rectangle



As with the association relationship, you can show navigability and multiplicity on an aggregation line. See "[Association](#)" for examples.

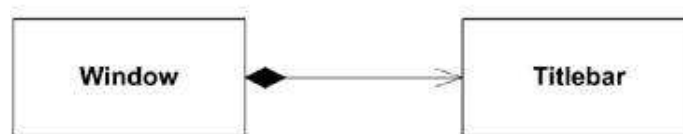
2.6.4. Composition

Composition represents a very strong relationship between classes, to the point of containment. Composition is used to capture a whole-part relationship. The "part" piece of the relationship can be involved in only one composition relationship at any given time. The lifetime of instances involved in composition relationships is almost always linked; if the larger, owning instance is destroyed, it almost always destroys the part piece. UML does allow the part to be associated with a different owner before destruction, thus preserving its existence, but this is typically an exception rather than the rule.

A composition relationship is usually read as "...is part of...", which means you need to read the composition from the part to the whole. For example, if you say that a window in your system *must* have a titlebar, you can represent this with a class named `Titlebar` that "...is part of..." a class named `Window`.

You show a composition relationship using a filled diamond next to the owning class and a solid line pointing to the owned class. [Figure 2-27](#) shows an example composition relationship between a class named `Window` and a class named `Titlebar`.

Figure 2-27. Titlebar "is a part of" Window



As with the association relationship, you can show navigability and multiplicity on a composition line. See "[Association](#)" for examples.

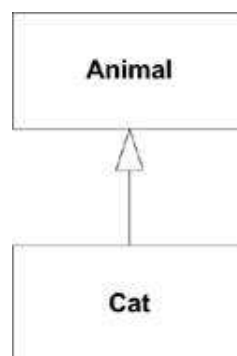
2.6.5. Generalization

A *generalization* relationship conveys that the target of the relationship is a general, or less specific, version of the source class or interface. Generalization relationships are often used to pull out commonality between different classifiers. For example, if you had a class named **Cat** and a class named **Dog**, you can create a generalization of both of those classes called **Animal**. A full discussion of how and when to use generalization (especially versus interface realization) is the subject for an object-oriented analysis and design book and isn't covered here.

Generalizations are usually read as "...is a...", starting from the more specific class and reading toward the general class. Going back to the **Cat** and **Dog** example, you would say "a **Cat**...is a...**Animal**" (grammar aside).

You show a generalization relationship with a solid line with a closed arrow, pointing from the specific class to the general class. [Figure 2-28](#) shows an example of the **Cat** to **Animal** relationship.

Figure 2-28. Cat specializes the Animal base class

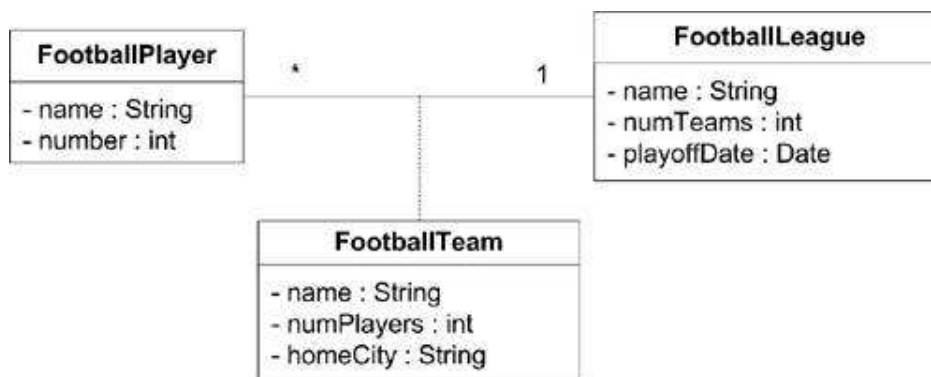


Unlike associations, generalization relationships are typically not named and don't have any kind of multiplicity. UML allows for multiple inheritance, meaning a class can have more than one generalization with each representing an aspect of the decedent class. However, some modern languages (e.g., Java and C#) don't support multiple inheritance; interfaces and interface realization are used instead.

2.6.6. Association Classes

Often the relationship between two elements isn't a simple structural connection. For example, a football player may be associated with a league by virtue of being on a team. If the association between two elements is complex, you can represent the connection using an *association class*. An association class is an association that has a name and attributes, like a normal class. You show an association class like a regular class with a dashed line connecting it to the association it represents. [Figure 2-29](#) shows a football player's relationships to a league.

Figure 2-29. Example association class

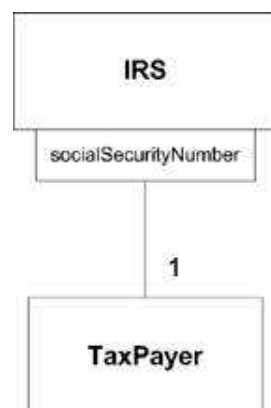


When translated into code, relationships with association classes often result in three classes: one for each end of the association and one for the association class itself. There may or may not be a direct link between the association ends; the implementation may require you to traverse through the association class to get to the opposite end of the link. In other words, `FootballPlayer` may not have a direct reference to `FootballLeague` but may have a reference to `FootballTeam` instead. `FootballTeam` would then have a reference to `FootballLeague`. How the relationships are constructed is a matter of implementation choices; however, the fundamental concept of an association class is unchanged.

2.6.7. Association Qualifiers

Relationships between elements are often keyed, or indexed, by some other value. For example, a bank patron may be identified by her account number, or a tax payer by his Social Security number. UML provides association *qualifiers* to capture such information. A qualifier is typically an attribute of the target element, though this isn't required. You show a qualifier by placing a small rectangle between the association and the source element. Draw the name of the qualifier (usually the name of an attribute) in the rectangle. [Figure 2-30](#) shows the relationship between the IRS and a taxpayer qualified by the taxpayer's Social Security number.

Figure 2-30. An association qualifier



Notice that the multiplicity between the association qualifier and `TaxPayer` is `1`. Obviously the `IRS` is associated with more than one `TaxPayer`, but by using qualifiers, you indicate that a `socialSecurityNumber` uniquely identifies a single `TaxPayer` within the `IRS`.

[< PREV](#)[NEXT >](#)

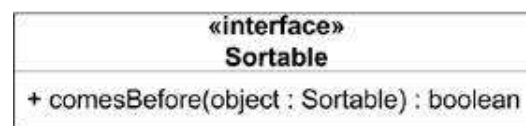
2.7. Interfaces

An *interface* is a classifier that has declarations of properties and methods but no implementations. You can use interfaces to group common elements between classifiers and provide a contract a classifier that provides an implementation of an interface must obey. For example, you can create an interface named `Sortable` that has one operation named `comesBefore(...)`. Any class that realizes the `Sortable` interface must provide an implementation of `comesBefore(...)`.

Some modern languages, such as C++, don't support the concept of interfaces; UML interfaces are typically represented as pure abstract classes. Other languages, such as Java, do support interfaces but don't allow them to have properties. The moral is that you should be aware of how your model is going to be implemented when modeling your system.

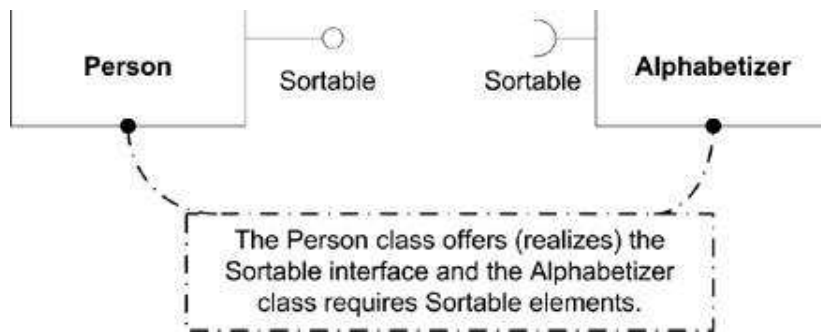
There are two representations for an interface; which one you should use depends on what you're trying to show. The first representation is the standard UML classifier notation with the stereotype `«interface»`. [Figure 2-31](#) shows the `Sortable` interface.

Figure 2-31. The Sortable interface



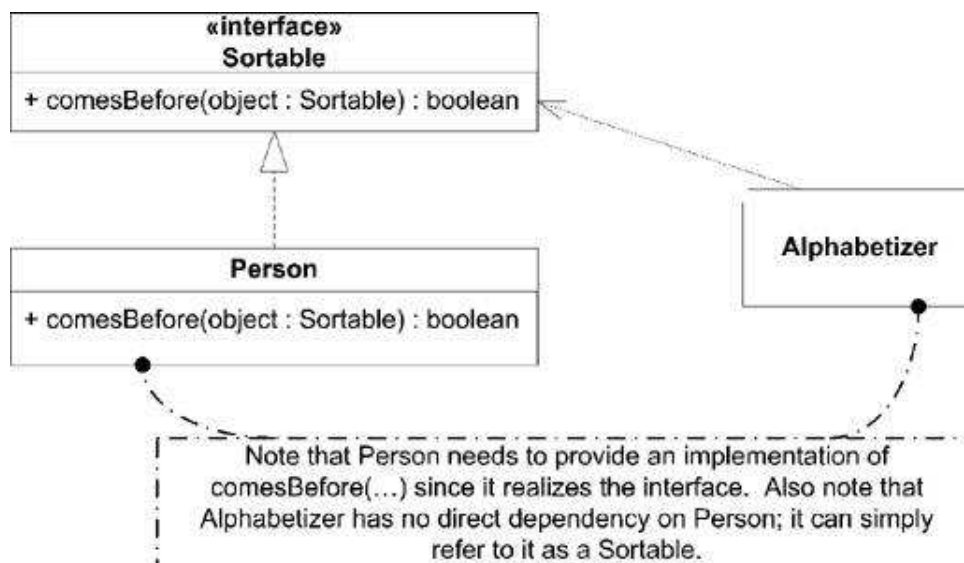
The second representation of an interface is the ball-and-socket notation. This representation shows less detail for the interface but is more convenient for showing relationships to classes. The interface is simply shown as a ball with the name of the interface written below it. Classes dependent on this interface are shown attached to a socket matching the interface. [Figure 2-32](#) shows the `Sortable` interface using the ball-and-socket notation.

Figure 2-32. Examples of providing and requiring interfaces



Because an interface specifies the contract only for a set of features, you can't instantiate an interface directly. Instead, a class is said to *realize* an interface if it provides an implementation for the operations and properties. You show realization using a dashed line starting at the realizing classifier and leading to the interface, with a closed arrowhead at the end. Classes that are dependent on the interface are shown using a dashed line with an open arrow (dependency). [Figure 2-33](#) shows a class that realizes the `Sortable` interface and a class that is dependent on it.

Figure 2-33. Person realizes the Sortable interface and Alphabetizer depends on it



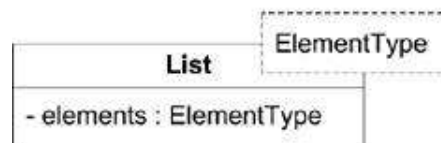
Providing an implementation of an operation is straightforward. You must provide an implementation on a realizing classifier with the same signature as the operation on the interface. Typically there are semantic constraints associated with an operation that must be honored by any implementation. Realizing a property is more subtle. A property on an interface states that any class that realizes the interface must store the data specified by the property *in some way*. A property on an interface doesn't necessarily mean there will be an associated property on a realizing classifier. However, the classifier must be able to store the data represented by the property and provide a means to manipulate it.

2.8. Templates

Just as interfaces allow you to provide specifications for *objects* your class will interact with, UML allows you to provide abstractions for the type of *class* your class may interact with. For example, you can write a `List` class that can hold any type of object (in C++ this would probably be a `void*`, in Java and C# it would probably be an `Object`). However, while you wanted your `List` class to be able to support any type of object, you want all of the objects in a given list to be of the same type. UML allows you to create and specify these kinds of abstractions using *templates*.

You can indicate that a class is a templated (also called *parameterized*) class by drawing a dashed rectangle in the upper-right corner of the class. For each element you would like to template, you need to specify a name to act as a placeholder for the actual type. Write the placeholder name in the rectangle. [Figure 2-34](#) shows an example of a `List` class that can support any type.

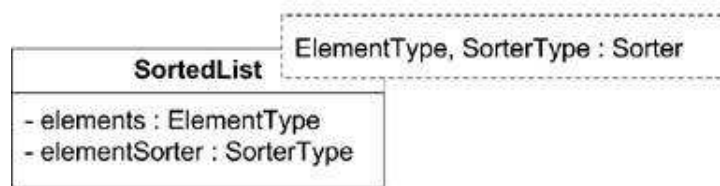
Figure 2-34. A templated List class



 This example uses `ElementType` as the name of the templated type for clarity. In practice, this is often abbreviated to just `T`.

You can have multiple templated types within a single class; just separate the type names with a comma (,). If you need to restrict the types the user may substitute, show that with a colon (:) followed by the type name. [Figure 2-35](#) shows a more complicated version of the `List` class that requires a `Sorter` along with the type of object to store in the list.

Figure 2-35. A templated class with type restrictions



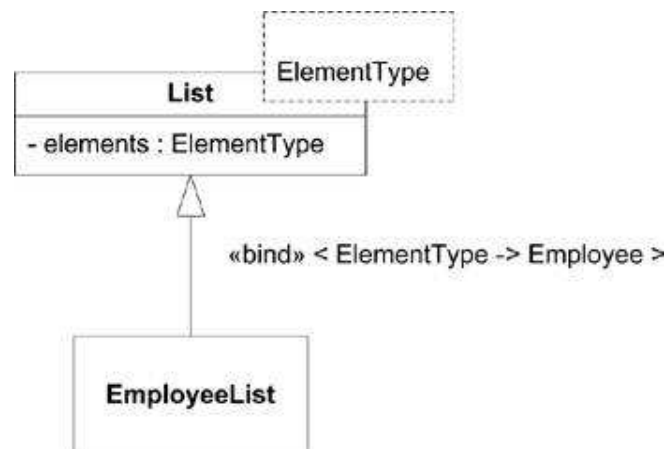
Specifying restrictions on a type that may be used is functionally similar to specifying an interface for a templated member, except that the user may be able to further restrict an instance of your class by specifying a subclass of your type.

When a user creates an instance of a `List`, she needs to specify the actual type to use in place of `ElementType`. This is called *binding* a type to a template. You show binding with the keyword `«bind»`, followed by a type specification using the following syntax:

```
<TemplatedType -> RealType >
```

You can use the binding syntax whenever you refer to a templated class to indicate you want to use a bound version of that class. This is called *explicit binding*. For example, [Figure 2-36](#) shows a subclass of `List` called `EmployeeList` that binds the `ElementType` of `List` to a class named `Employee`.

Figure 2-36. Explicit template binding



The `bind` keyword also indicates what types should be used with an instance of a template. This is called *implicit binding* and is shown in [Figure 2-37](#).

Figure 2-37. Implicit template binding

ExampleList : List <ElementType->Employee>

◀ PREV

NEXT ▶

2.9. Variations on Class Diagrams

Because class diagrams model structures well, they can be used to capture well-defined, hierarchical information. Class diagrams have been used to capture XML and database schemas with some degree of success. In the interest of full disclosure, people who deal with XML and database schemas extensively have reservations about using class diagrams to capture the information, specifically because class diagrams are so generic. Each domain has its own notation that may be better suited to capturing complex relationships or domain-specific information. However, UML has the benefit of being a common language that is understood by those outside the XML and database domains.

2.9.1. XML Schemas

The structural design of an XML document can be captured in an XML schema. XML schemas are to XML documents as classes are to objects; XML documents are instances of a schema. Therefore, it's not a giant leap to realize that class diagrams can be used to model XML schemas. XML schemas are described using the XML Structure Definition Language (XSDL). XSDL is a text language (as opposed to the graphical nature of UML) and can be verbose; mapping XSDL to UML can make it much easier to digest a schema document.

The fundamental elements of XSDL are XML elements, which are connected in sequences, choices, and complex structures. Each element may have extra information attached to it using (conveniently enough) attributes. Modelers typically represent XML elements as classes and XSDL attributes as UML attributes. Each element is linked to the next using composition arrows. Multiplicity specifications on the relationships show how many times one element appears within another. [Example 2-1](#) is a sample XML document that describes a piece of equipment.

Example 2-1. A sample XML document

```
<?xml version="1.0" encoding="UTF-8"?>
<equipmentlist xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="equipment.xsd">
  <equipment equipmentid="H-1">
    <shortname>
      <langstring lang="en">Hammer</langstring>
    </shortname>
    <technicalcontact>
      <contact>
        <name>Ron</name>
        <telephone>555-1212</telephone>
      </contact>
    </technicalcontact>
    <trainingcontact>
      <contact>
        <name>Joe</name>
        <email>joe@home.com</email>
      </contact>
    </trainingcontact>
  </equipment>
</equipmentlist>
```

```

        </trainingcontact>
    </equipment>
</equipmentlist>

```

[Example 2-2](#) is the XSDL that describes the XML document.

Example 2-2. XML schema describing [Example 2-1](#)

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
    elementFormDefault="qualified">

    <!-- ~Class: contact ~~~~~~ -->
    <xs:element name="contact" type="contact"/>
    <xs:complexType name="contact">
        <xs:sequence>
            <xs:element name="name" type="xs:string"/>
            <xs:choice>
                <xs:element name="telephone" type="xs:string"/>
                <xs:element name="email" type="xs:string"/>
            </xs:choice>
        </xs:sequence>
    </xs:complexType>

    <!-- ~Class: equipment ~~~~~~ -->
    <xs:element name="equipment" type="equipment"/>
    <xs:complexType name="equipment">
        <xs:sequence>
            <xs:element ref="shortname"/>
            <xs:element name="technicalcontact" type="technicalcontact"/>
            <xs:element name="trainingcontact" type="trainingcontact"/>
        </xs:sequence>
        <xs:attribute name="equipmentid" type="xs:string" use="required"/>
    </xs:complexType>

    <!-- ~Class: equipmentlist ~~~~~~ -->
    <xs:element name="equipmentlist" type="equipmentlist"/>
    <xs:complexType name="equipmentlist">
        <xs:sequence>
            <xs:element ref="equipment" minOccurs="1" maxOccurs="unbounded"/>
        </xs:sequence>
    </xs:complexType>

    <!-- ~Class: <<XSDDtopLevelAttribute>> lang ~~~~~~ -->
    <xs:attribute name="lang" type="xs:language"/>

    <!-- ~Class: langstring ~~~~~~ -->
    <xs:element name="langstring" type="langstring"/>
    <xs:complexType name="langstring">
        <xs:simpleContent>
            <xs:extension base="xs:string">
                <xs:attribute name="lang" use="required">
                    <xs:simpleType>
                        <xs:restriction base="xs:NMTOKEN">

```

```

        <xs:enumeration value="en"/>
        <xs:enumeration value="fr"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:attribute>
</xs:extension>
</xs:simpleContent>
</xs:complexType>

<!-- ~Class: shortname ~~~~~~ -->
<xs:element name="shortname" type="shortname"/>
<xs:complexType name="shortname">
  <xs:sequence>
    <xs:element ref="langstring" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<!-- ~Class: technicalcontact ~~~~~~ -->
<xs:element name="technicalcontact" type="technicalcontact"/>
<xs:complexType name="technicalcontact">
  <xs:sequence>
    <xs:element ref="contact" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<!-- ~Class: trainingcontact ~~~~~~ -->
<xs:element name="trainingcontact" type="trainingcontact"/>
<xs:complexType name="trainingcontact">
  <xs:sequence>
    <xs:element ref="contact" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
</xs:schema>

```

[Figure 2-38](#) is a UML representation of the schema. Elements are represented as classes, with one exception: the `contact` definition includes a `choice` option. [Figure 2-38](#) represents this using another class stereotyped as `XSDchoice`, and the options are represented as attributes of the class.

So, while UML simplifies a schema by representing the information graphically, details such as sequencing and type information can be lost without using a UML extension mechanism such as constraints or stereotypes.

2.9.2. Database Schemas

By mapping database tables to classes and table rows to attributes, you can capture a database schema fairly well using UML. Additional information such as primary keys, foreign keys, and constraints can be captured using UML constraints or stereotypes. You can show relationships between tables using associations between classes (usually composition relationships, but that is just by convention). [Figure 2-39](#) shows a sample database schema represented using a UML class diagram.

As with XML schemas, few professional database administrators model databases using class diagrams. UML class diagrams are useful for conveying schema information in a common

language, but they lack the expressive (and custom) capabilities of the more standard database notation, Entity Relation Diagrams (ERDs).

Figure 2-38. A class diagram representation of [Example 2-2](#)

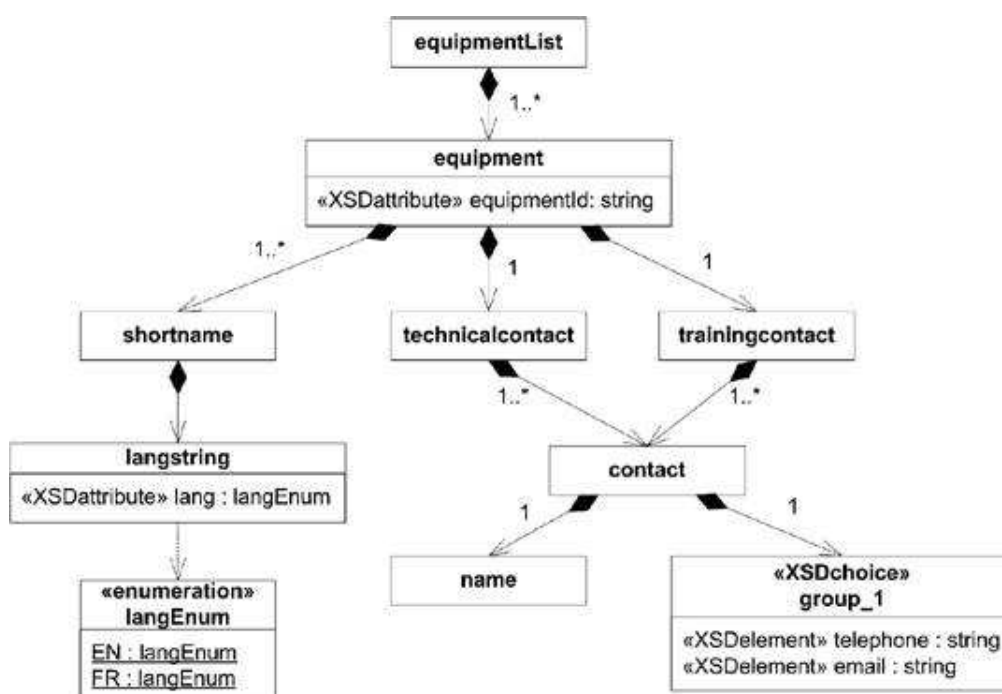
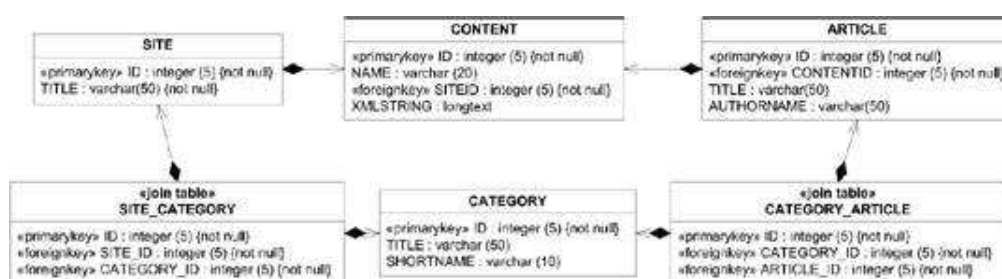


Figure 2-39. An example database schema using a class diagram



[< PREV](#)[NEXT >](#)

Chapter 3. Package Diagrams

Packages provide a way to group related UML elements and scope their names. For example, you can put all elements having to do with 3D rendering into a package named `3DGraphics`. Package diagrams provide a great way to visualize dependencies between parts of your system and are often used to look for problems or determine compilation order.

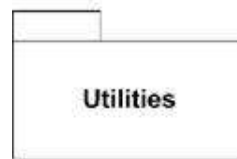
Nearly all UML elements can be grouped into packages, including packages themselves. Each package has a name that scopes each element in the package. For example, if you had a class named `Timer` in a package named `Utilities`, the fully qualified name for the class is `Utilities::Timer`. Elements in the same package can refer to each other without qualifying their names.

[< PREV](#)[NEXT >](#)

3.1. Representation

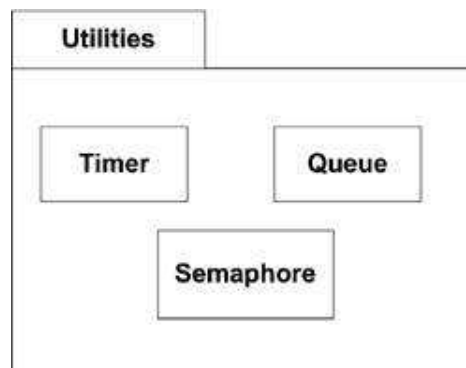
You show a package using a rectangle with a tab attached to the top left. [Figure 3-1](#) shows the `Utilities` package.

Figure 3-1. A simple package



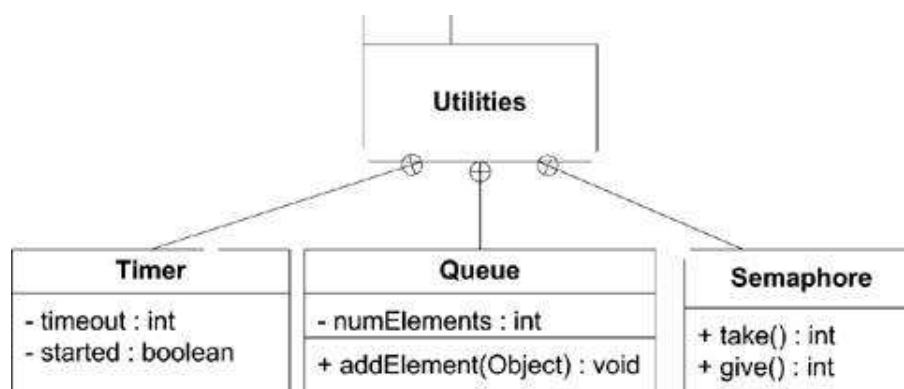
You can show the elements contained within a package in two different ways. First, you can show the elements contained within the package by drawing them inside the large rectangle. If you use this representation, write the name of the package in the tab. [Figure 3-2](#) shows the contents of the `Utilities` package inside of the package. To refer to the `Timer` class from outside of the `Utilities` package, you say `Utilities::Timer`.

Figure 3-2. Several classes contained inside the Utilities package



The second representation uses a solid line pointing from the package to each contained element. You place a circle with a plus sign in it at the end nearest the package to indicate containment. If you use this representation, you should show the name of the package in the large rectangle rather than in the tab. This notation allows you to show more detail of the packaged elements. [Figure 3-3](#) shows the same `Utilities` package but with the classes broken out.

Figure 3-3. Elements owned by a package, shown outside of the package



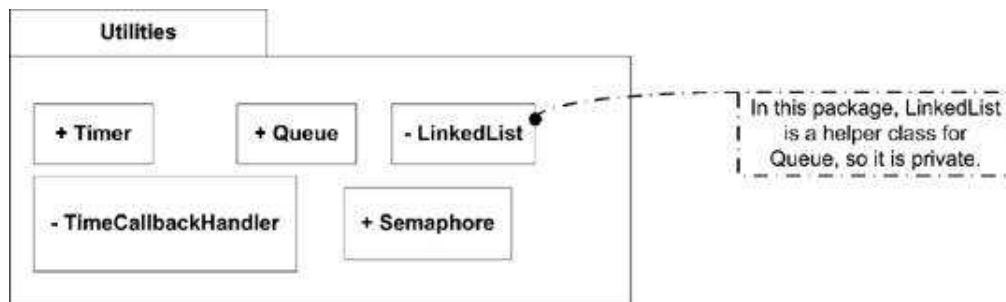
You don't need to show all the elements contained within a package; if no elements are shown, no assumptions can be made about what the package contains.

3.2. Visibility

A package may specify visibility information for owned and imported elements, however elements may have only one of two levels of visibility: public or private. Public visibility means the element may be used outside the package (scoped by the package name). Private visibility means the element may be used only by other elements of the same package. Private visibility is useful for marking utility classes that help implement a subsystem or component you don't want to expose to the rest of the system.

You show public visibility by placing a plus sign before the element name. You show private visibility using a minus sign. [Figure 3-4](#) shows the `Utility` package with some private helper classes.

Figure 3-4. The Utilities package with public and private members



3.3. Importing and Accessing Packages

When accessing elements in one package from a different package, you must qualify the name of the element you are accessing. For example, if `Car` is a class in the `transportation` package and you are trying to access it from a package named `RoutePlanning`, you need to qualify `Car` as `transportation::Car`.

To simplify accessing elements in a different package, UML allows a package to *import* another package. Elements of the imported package are available without qualification in the importing package. So, if the `RoutePlanning` package imported the `transportation` package, you can refer to `Car` without any qualifications from within the `RoutePlanning` package.

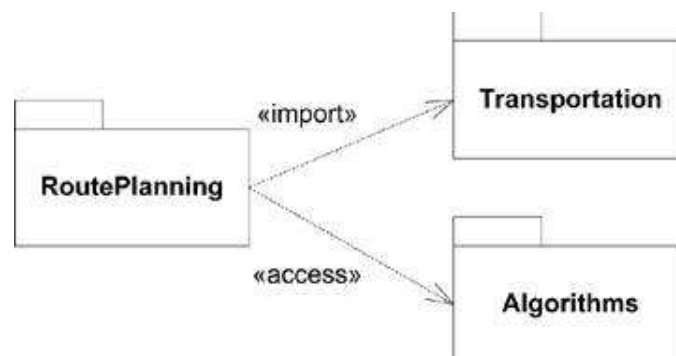
To show a package import, you draw a dashed line with an open arrow from the importing package to the imported package. Label this line with the `«import»` keyword. [Figure 3-5](#) shows the `RoutePlanning` package importing the `transportation` package.

Figure 3-5. RoutePlanning importing the Transportation package



By default, imported elements are given public visibility in the importing package. UML allows you to specify that imported elements should be given private visibility, meaning they can't be used by anyone outside the importing package (including any packages that may import that package). To specify that imported elements should have private visibility, you use the `«access»` keyword rather than the `«import»` keyword. [Figure 3-6](#) shows the `RoutePlanning` package importing the `TRansportation` package and accessing the `Algorithms` package. If a package imports the `RoutePlanning` package, both packages can use public elements from `TRansportation`, but they can't use anything in `Algorithms`.

Figure 3-6. RoutePlanning importing the Transportation package but accessing only the Algorithms package



What package import and access actually mean in implementation can vary dramatically depending on your target language. For example, C# and Java have an explicit concept of packages and importing elements from packages. Java developers often import the `java.util` package into their program so they can reference the Java `Vector` class without qualifying it. However, C++ has a somewhat subtler concept of packages, called namespaces. How packages map to an implementation language is often up to the implementer.

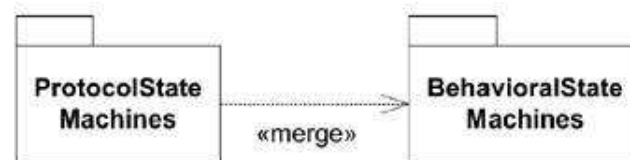
3.4. Merging Packages

UML supports a somewhat complex concept of merging packages. Merging packages differs from importing packages in that *merge*, by definition, creates relationships between classes of the same name. The motivation behind package merging comes directly from the evolution of UML from 1.x to 2.0. UML 2.0 defines the base concept of elements and allows specific diagram types to extend a base concept without needing to provide a new name for it. For example, UML extends several core Behavioral State Machine concepts into Protocol State Machine concepts while retaining their original name.

When a package merges another package, any class of the same type and name automatically extends (or has a generalization relationship to) the original class. For example, UML can define the concept of the *include* relationship at a generic level and then specialize it for use cases inclusion and retain the name *include*. This type of extension has simplified the internals of the UML model but rarely makes an appearance in real-world development.

You show package merging using a dashed line with an open arrow from the merging package to the merged package. Label this line with the keyword `«merge»`. [Figure 3-7](#) shows an example of merging two packages.

Figure 3-7. ProtocolStateMachines merging the elements from BehavioralStateMachines so it can add to the contained classes



The rules for package merge are:

- Private members of a package aren't merged with anything.
- Classes in the package performing the merge that have the same name and type as classes in the merged package(s) get a generalization relationship to the merged class(es). Note that this can result in multiple inheritance, but UML allows this.
- You can still reference any of the original classes by explicitly scoping the class using the original package name.
- Classes that exist only in the merged package or the merging package remain unchanged and are added to the merging package.
- Subpackages within the merged package are added to the merging package if they don't already exist.

- If a subpackage with the same name already exists in the merging package, another merge is started between the two subpackages.
- Any package imports from the merged package become package imports from the merging package. Elements that are imported aren't merged (i.e., aren't given generalization relationships). If an imported element conflicts with an element in the merging package, the merging package's element takes precedence and the imported element must be explicitly qualified.

[◀ PREV](#)[NEXT ▶](#)

3.5. Variations on Package Diagrams

This section presents two applications of class package diagrams and one application of use case packages. Although layering and simplification always motivate packaging, the term "simplification" means different things to different people. Simplification can mean:

- Easier to build and test
- Better tracking and project transparency
- Working at a stable overview without the noise of low-level churn
- Less conflict between distributed teams
- Easy refactoring and extension

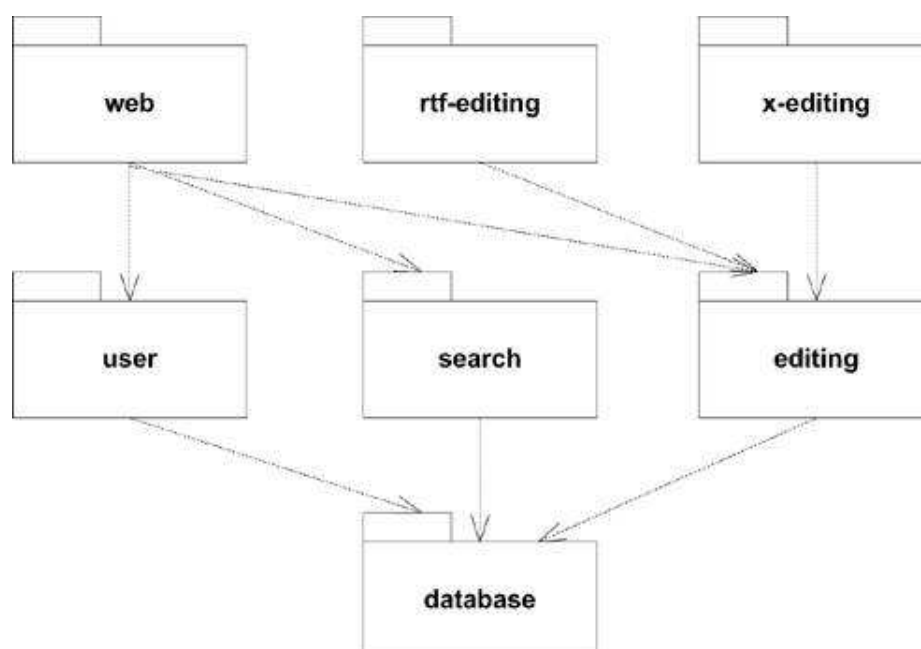
Simplification likely means more apparent complexity to some constituency. Unless your packaging balances these diverse needs, you are likely to receive complaints of unnecessary complexity, no matter how noble your motives are.

3.5.1. Structuring a Project with Package Diagrams

Class packages organize a logical system during construction. They provide the terms for management and external stakeholders, as well as structure to contain all classes and code to be built. Class package diagrams are the UML equivalent of block diagrams.

Different parties think of a project according to their different needs. Programmers think in terms of language and tactical design. Architects think in terms of dependencies, risk, technology, building, testing, and OO principles. Project managers think in terms of risk, tracking, resources, need, ownership, required skills, and delivery. Although all issues are important, and good packaging recognizes its responsibility to all needs, architects tend to identify top-level packaging with an eye on the control functions of project management. For example, a project manager might draw a package diagram for a web application such as that shown in [Figure 3-8](#).

Figure 3-8. Top-level packages defining a web application



The diagram in [Figure 3-8](#) carries very little meaning by itself. It must be accompanied by an auxiliary, textual document that describes the basis for the packaging. Such a document might, for example, contain a list as follows:

web

Requires special skills: HTML, CSS, and Struts, a presentation technology; most dependencies.

database

Requires database management and modeling skills; most independent/fewest dependencies.

user

To be built off-site by a remote team.

search

Requires familiarity with search engine technology and techniques; self-contained subsystem.

editing

Comprises the basic editing features to be delivered in the first release; different skills and different team.

rtf-editing

Comprises those editing features scheduled for release 2.

x-editing

Comprises editing features requested by a specific client. These features to be withdrawn or delayed depending on that client. Risk is independent of the other features.

[Figure 3-8](#) doesn't show the complete set of packages in the system. It is merely enough for management to track and control the project. Programmers create lower-level packages as needed for correct code design. Managers, however, allocate resources and track progress on the relatively large grains of the project shown in [Figure 3-8](#) without getting disturbed by the addition or refactoring of internal packages and contents.

3.5.2. Use Case Packages

Use case packages organize the functional behavior of a system during analysis. [Figure 3-9](#) shows the major functional areas of a content management system. The packages provide understandable terms for team members outside the analyst team. Managers can discuss the project at an appropriate level of detail without getting bogged down in details.

The CMS system shown in [Figure 3-9](#) comprises packages created for the following reasons:

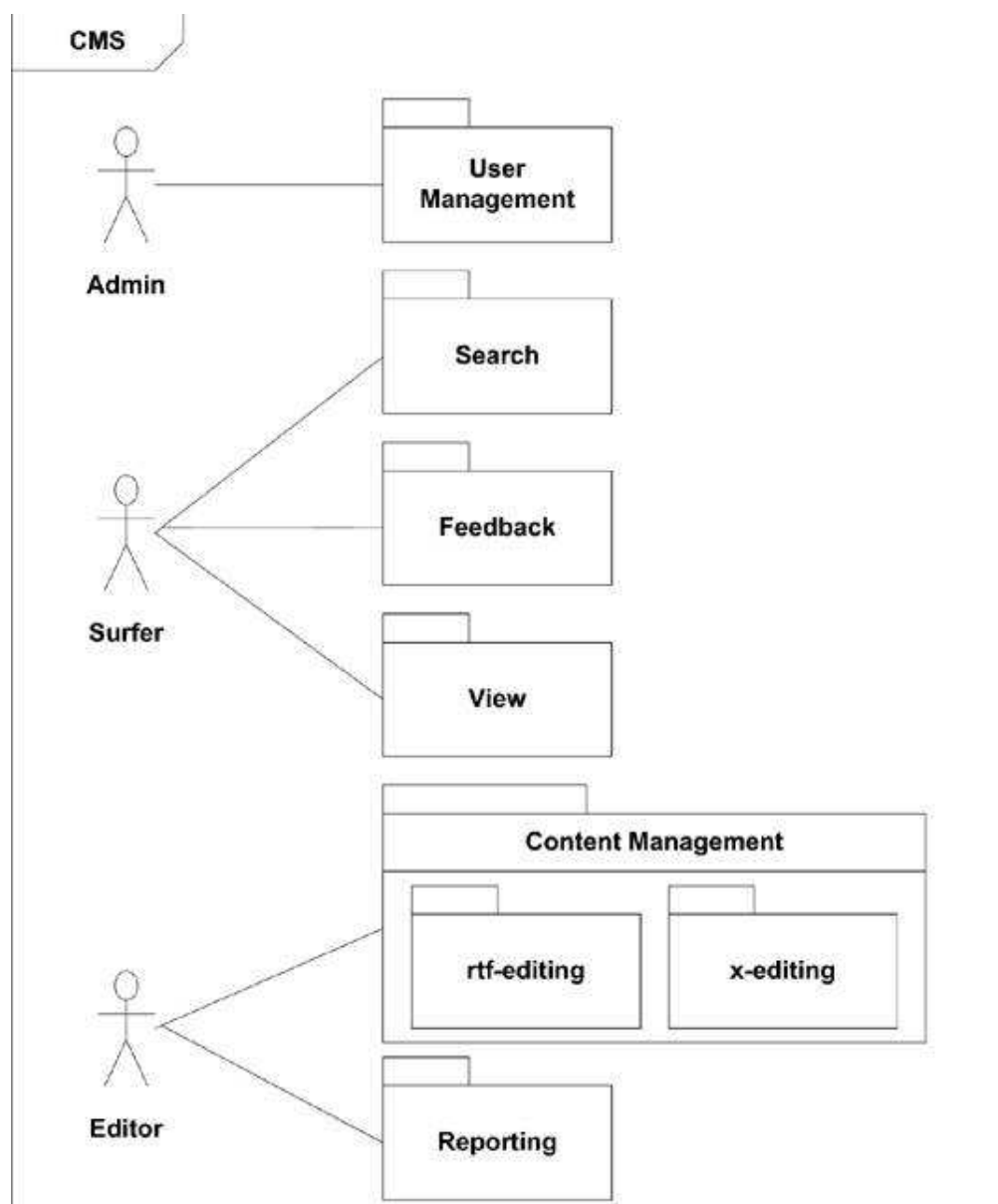
- The complex editing packages separate the preliminary delivery and the advanced delivery from the client-based delivery.
- The simple interactions, view and feedback, contain basic, low-risk functions.
- The reporting has more sophisticated functions.
- The search and the user management separate complex functions.

Just as with the class packages in [Figure 3-8](#), you manage the use case packages to manage the project. Remember that tracking use case packages tracks customer value.

3.5.3. Directed Dependency Graphs

Directed dependency graphs of the packages in a system reveal nonfunctional issues related to buildability, testability, and robustness; they provide a guide for several roles in software development.

Figure 3-9. A set of functional major use case packages

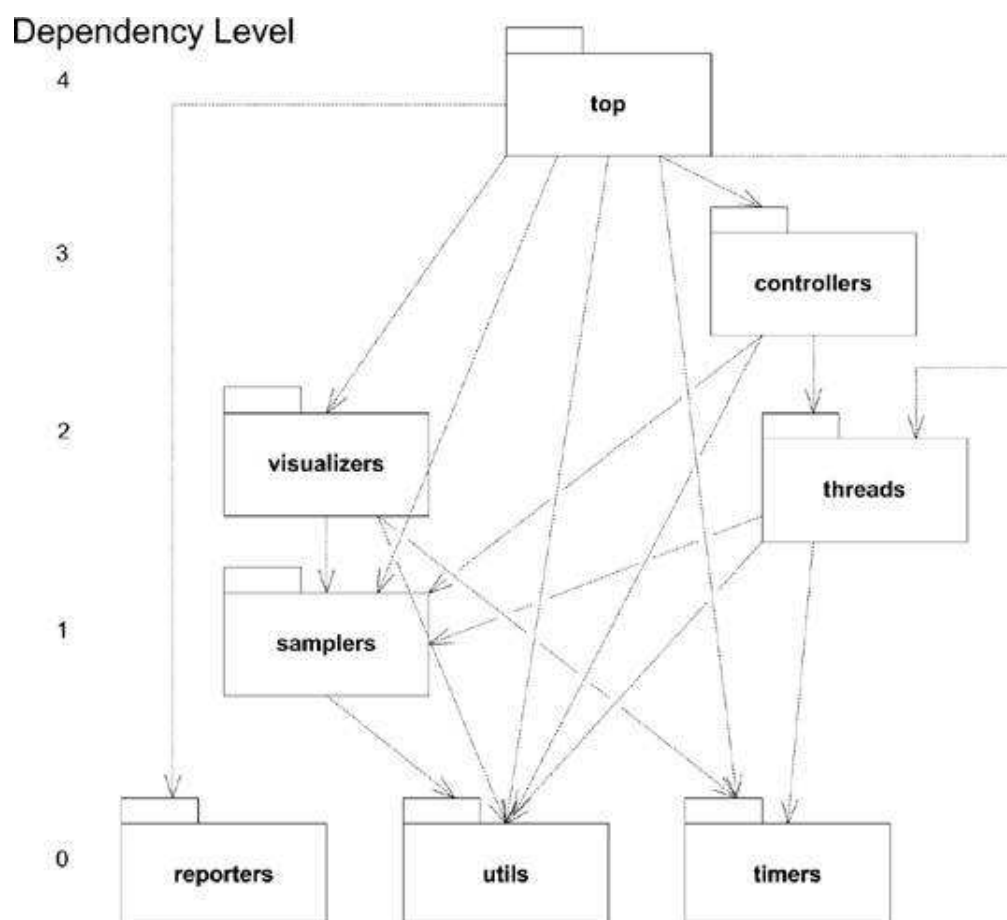


When all the dependencies flow in one direction, as in [Figure 3-10](#), without any loops or cycles, the graph is *acyclic*. Acyclic graphs show a project in good health.

Directed dependency graphs can help you avoid problems with build scripts and project testing. Looking at [Figure 3-10](#), you can see that refactoring `timer` will invalidate testing and potentially break the build of `visualizers`, `threads`, `controllers`, and `top`; you should thus change the `timer` package with caution.

Directed dependency graphs can also help you divide project work among different staff members. For example, [Figure 3-10](#) shows no dependencies between `threads` and `visualizers`. Thus, different groups of people can work on those two packages without destabilizing each other.

Figure 3-10. Dependency graph of modified JMeter with no cycle



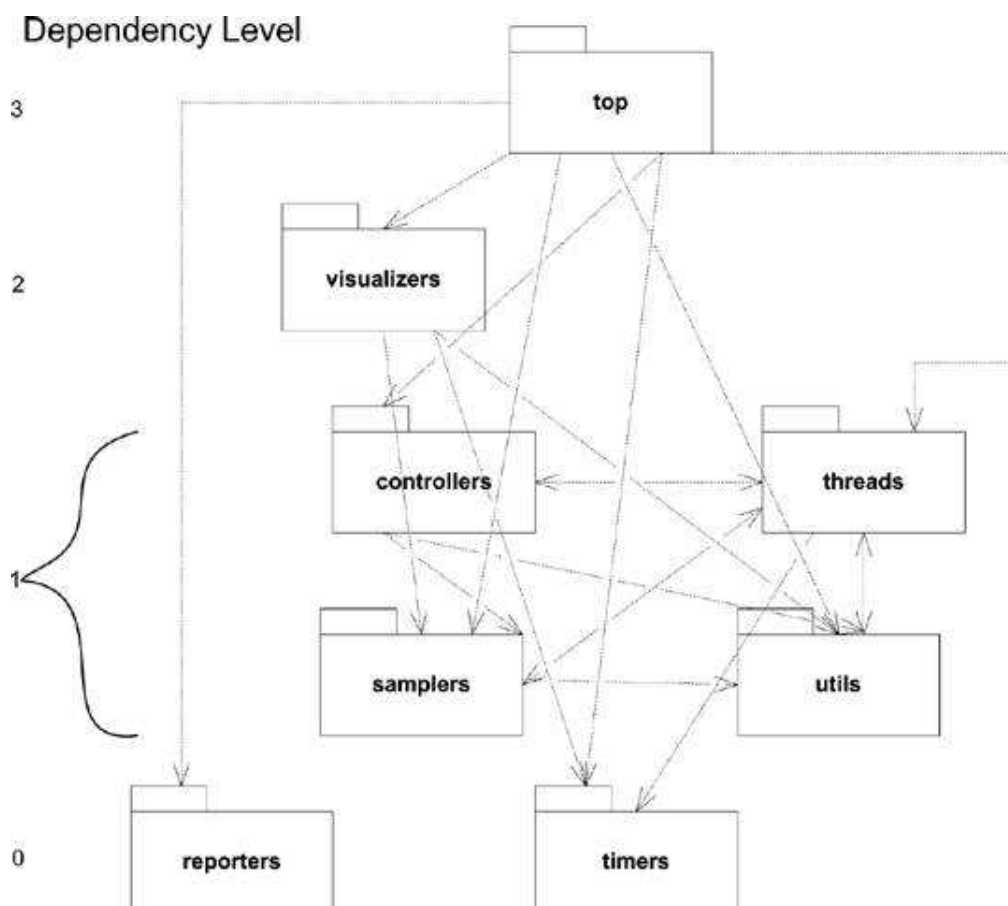
Through time, projects evolve, and new dependencies creep in because of misunderstanding or expediency. [Figure 3-11](#) shows such a case; three bidirectional dependencies ruin the desirable, acyclic nature of the graph:

- `utils` now depends on `threads`.
- `threads` now depends on `controllers`.
- `samplers` now depends on `threads`.

The problem here is that the reverse of each dependency already existed in [Figure 3-10](#). The result is the tangle of bidirectional arrows in level 1 of [Figure 3-11](#), which contrasts with the clear flow of arrows in [Figure 3-10](#). The further result is pain, for the developers, testers, and likely, managers and users. Changes in `threads` will confound the team responsible for `controllers` and `utils`, and vice versa because fixes in one require fixes in the code or unit tests of the others. It may take several iterations until all packages stabilize.^[*]

[*] For more theory and metrics regarding packages and their dependencies, see John Lakos' *Large-Scale C++ Software Design*. Also see Robert C. Martin's <http://c2.com/cgi/wiki?PrinciplesOfObjectOrientedDesign>.

Figure 3-11. Dependency graph of modified JMeter with three cycles



[◀ PREV](#)[NEXT ▶](#)

Chapter 4. Composite Structures

As a system becomes more complex, it is often helpful to decompose it in terms of functionality (see [Chapter 7](#)). To realize a piece of functionality, different elements of a system often work together and communicate information. UML 2.0 formalizes the concept of complex relationships between elements into the idea of composite structures. Much of the material in this chapter is new to UML 2.0.

[◀ PREV](#)[NEXT ▶](#)

4.1. Composite Structures

A *structure* is a set of interconnected elements that exist at runtime to collectively provide some piece of functionality. For example, you can use a structure to represent the internal makeup of a classifier such as a subsystem (what objects are related to each other, who is communicating with whom, etc.). UML calls such structures *internal structures*. UML defines several symbols to capture the relationships and communications between elements in an internal structure.

4.1.1. Connectors

Connectors represent communication links between instances of classes participating in an internal structure. They can be runtime instances of associations, or they can represent dynamic communication set up at runtime for example by being values of local variables. You show a connector as a solid line between two instances. Note that while associations between classes represent a link between any instances of those classes, a connector represents a link between only the two instances specified at each end of the connector (see "[Collaborations](#)").

You can provide name and type information for a connector using the following format:

name : *classname*

where:

name

Is the name of the connector. The name can be used later in collaborations to reference this connector.

classname

Is the name of an association this connector represents.

[Figure 4-1](#) is an example of a named connector.

Figure 4-1. The link between a Pedal and a Wheel is a connector named "c," which is a Chain

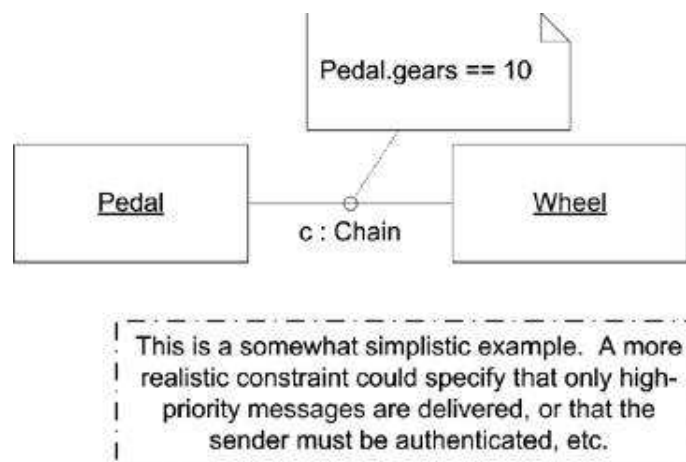


UML specifies several rules for determining the types of the elements at each end:

- If a connector is an instance of an association, the types of the instances at either end of the connector must be the same types at either end of the association.
- If an instance at one end of a connector has required interfaces, the instance at the other end of the connector must provide one of those interfaces.
- If an instance at one end of a connector has required interfaces and a port is connected to the other end of the connector, the port must provide a required interface.

The actual means of communication isn't specified by the connector; it can represent a simple method call or a complex protocol over a socket connection. You can constrain the connection using the normal UML note notation. Simply specify the constraint in a note, and link it to the connector using a dashed line. [Figure 4-2](#) shows a constrained connector.

Figure 4-2. A constrained connector



You may specify the multiplicity of each *connector end* using the normal multiplicity syntax. Simply write the number of instances for a given end near the end of the connector. For example, a student/teacher relationship may require at least 5 students and no more than 20. You can show the student/teacher relationship as depicted in [Figure 4-3](#).

Figure 4-3. A Teacher must have at least 5 Students but no more than 20



4.1.2. Ports

A *port* is a way to offer functionality from a composite structure without exposing the internal details of how that functionality is realized. For example, you may have a subsystem that can perform credit card payment verification. The actual implementation of this functionality may be spread over several classes working in conjunction. The organization of these classes can be represented as an internal structure within the subsystem, and the overall functionality, or credit card verification, can be exposed using a port. Exposing the functionality through a port allows the subsystem to be used by any other classifier that conforms to the port's specifications. Starting with UML 2.0, classes have been extended to allow for ports and internal structures. By default, ports are public, however UML 2.0 allows you to have internal ports that are available only to the composite structure hosting them.

You show a port as a small square. You typically draw the name and multiplicity of the port near the square, though both may be hidden. If you draw the port on the edge of a classifier, the port is public and is available to the environment. If you draw the port inside a classifier, the port is protected and available only to the composite structure. [Figure 4-4](#) shows an example port.

Figure 4-4. The CreditCardVerificationSubsystem with a single public port, CreditCardServices

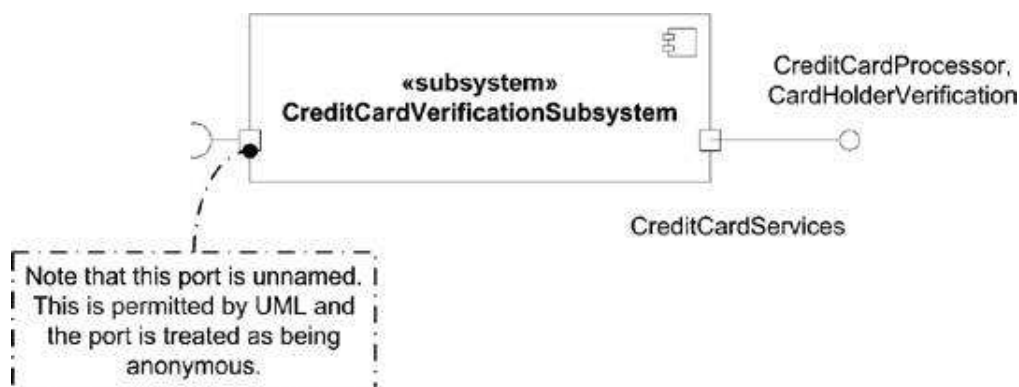


4.1.2.1. Required and provided interfaces

Ports are associated with required and provided interfaces (see "[Interfaces](#)" in [Chapter 2](#)). Required interfaces show what the owning classifier may ask of its environment through a given port. Provided interfaces show what functionality a classifier exposes to the environment. For example, our credit card payment system may provide an interface to verify credit cards, *CreditCardProcessor*, while requiring access to an account verification system, *AccountServices*, offered by the credit card company. If you use ports in your diagrams, the required and provided interfaces capture *all* the interaction the system may have with a given classifier. Provided and required interfaces are typically shown using the ball and socket (lollipop) notation, though you may explicitly type a port (see "[Port typing](#)"). If there are

multiple required or provided interfaces, simply list each interface name followed by a comma; don't show multiple lollipops. [Figure 4-5](#) shows a port with a required interface and two provided interfaces.

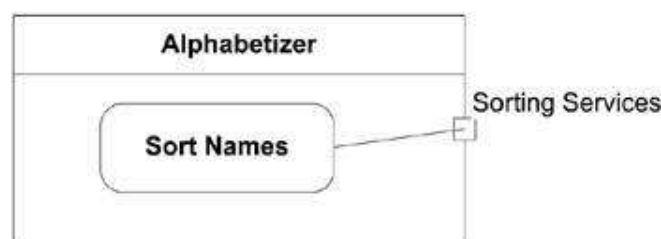
Figure 4-5. The CreditCardVerificationSubsystem providing two interfaces (CreditCardProcessor and CardHolderVerification) and requiring one (AccountServices)



4.1.2.2. Realizing port implementations

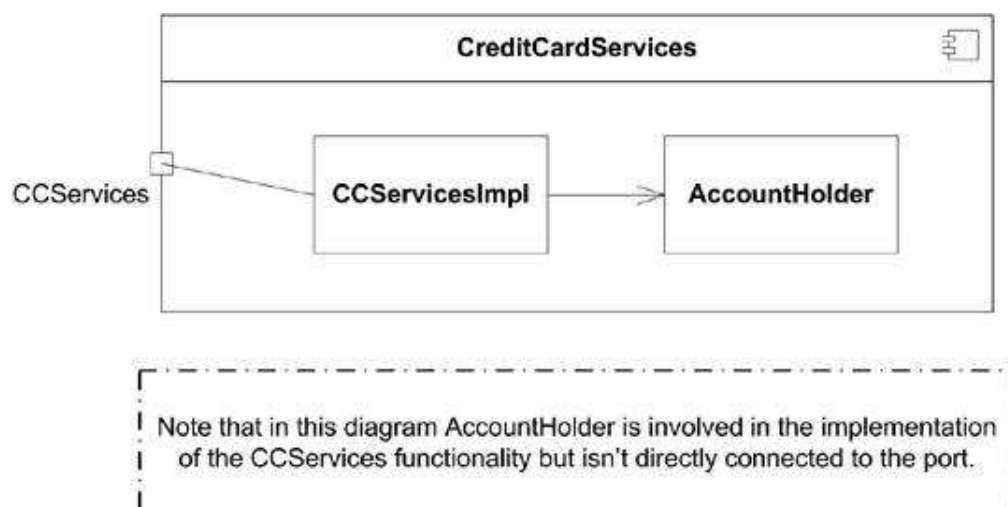
Ports are wired to an internal implementation using connectors. See "[Connectors](#)" for information on how to represent a connector. If the classifier owning the port provides the implementation of the functionality itself, the port is considered a *behavioral port*. In this case, the connector links to a state inside the classifier. This state is used to explain the behavior of the classifier when the port is used (see [Chapter 8](#) for more information on using states to model behavior). This is typically used for simple classifiers (not complex structures) that implement functionality themselves. [Figure 4-6](#) shows a behavioral port.

Figure 4-6. A behavioral port



On the other hand, if the functionality is realized by internal elements, you link the connector to internal classifiers that provide the implementation. This is typically used for composite structures such as components and subsystems. [Figure 4-7](#) shows such a port.

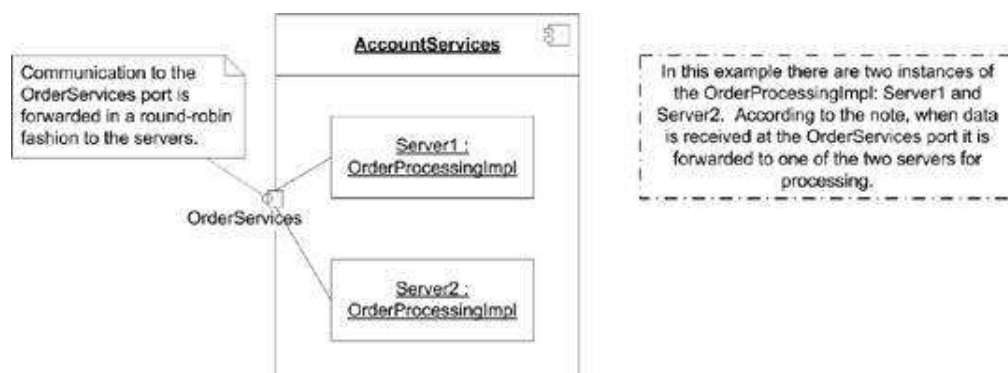
Figure 4-7. A port linked to an internal implementation



4.1.2.3. Multiple connectors

UML 2.0 allows you to have multiple connectors leading from a port to different internal elements. However, it doesn't specify what happens when communication is received at that port; this is left up to the modeler. Some possible solutions are to forward the communication to *all* connectors, forward based on priority, forward on a round-robin basis, or simply randomly choose a connector. Regardless of your decision, be sure to document it in your model, probably using a note attached to the port. [Figure 4-8](#) shows an example of using a port with many connectors.

Figure 4-8. A port with multiple connectors

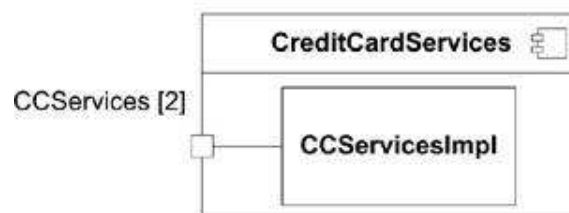


4.1.2.4. Port multiplicity

A classifier may specify multiplicity for a port like any other element. Simply place the desired number of port instances in brackets after the port name and type (if present). When the classifier is instantiated, the associated ports are instantiated as well. These are called *interaction points* and can be uniquely identified by the classifier. For example, if your classifier has two ports, one with provided interfaces that offer anonymous access to data and one with a provided interface that offers authenticated access to data, your classifier can

distinguish which port was used by the external system. [Figure 4-9](#) shows the credit card verification system offering two instances of the credit card verification port.

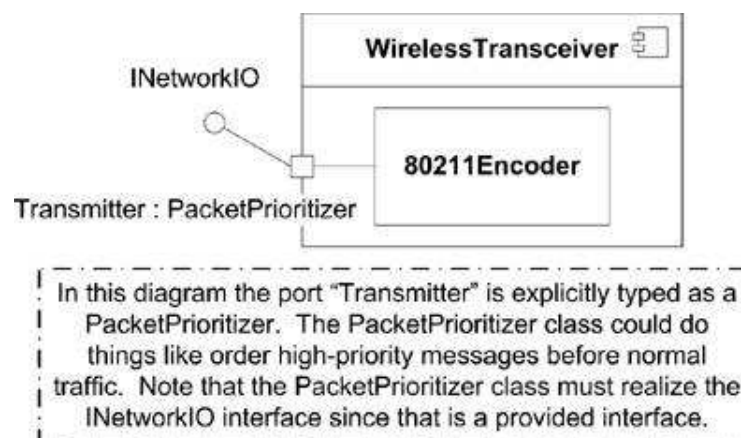
Figure 4-9. A component with two instances of the CCServices port



4.1.2.5. Port typing

In practice, when a port is instantiated, it is represented by a classifier that realizes the provided interfaces. Any communication with this interaction point simply passes the information to internal classifiers that realize the behavior. UML 2.0 allows you to specify the type of a port using classes to provide more sophisticated behavior. For example, you can specify that the port is typed using a class that filters the communications it receives or prioritizes delivery. When this port is instantiated, the corresponding class has a chance to manipulate the communication it receives before passing it to the realizing classifiers. To show that a port should be represented using a specific classifier, simply follow the name of the port with a colon and the name of the classifier to use. Note that the classifier must realize the provided interfaces. You can use this notation to show provided interfaces by using an interface as the port type, though the lollipop notation is often more flexible. [Figure 4-10](#) shows an explicitly typed port.

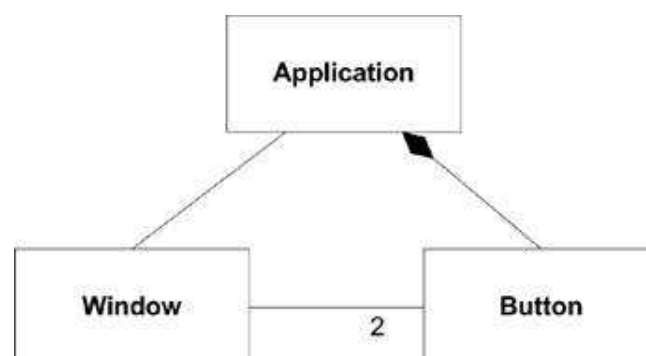
Figure 4-10. A port explicitly typed as a PacketPrioritizer



4.1.3. Structured Classes and Properties

As described in [Chapter 3](#), classifiers with whole-part relationships typically use composition arrows to show their relationships. In the context of composite structures, UML 2.0 has defined the term *property* to describe the "part" piece of the whole-part relationship. For example, in a graphical operating system, an application may be made up of a main window and several buttons. The buttons are a part of the application, so that the whole-part relationship between the application and the buttons can be shown using composition. However, the main window is shared with the operating system (so the system can reposition the window or hide it); as a result, the operating-system-to-window relationship is slightly weaker and is shown using association. You can model the application, window, and button relationships as shown in [Figure 4-11](#).

Figure 4-11. The Application, Window, Button relationship



When used in composite structure diagrams, relationships between properties are shown in the owning classifier's rectangle. This allows you to further restrict the association between parts of the composite classifier. For example, [Figure 4-11](#) shows that Buttons can be associated with any Window. Using a composite structure diagram, you can restrict the Window to be associated only with a Button owned by the same application. When you draw the composite structure diagram, properties that are associated with the composite structure through composition are shown with solid rectangles, and properties that are shared with other structures are shown using a dashed rectangle. You may place multiplicity information for a property in its rectangle in the upper right corner, or after the name of the property in brackets. [Figure 4-12](#) shows the composite structure diagram of application, window, and button relationships.

In addition to simply specifying how properties fit together, you can use composite structures to specify how instances are instantiated. As in [Chapter 3](#), instances are shown by underlining the name and type of the classifier. You may specify initial values for each attribute of a classifier by specifying the name of the attribute followed by an equals sign and the value of the attribute. When used as instances, you can specify the roles each property will take on by showing a slash "/" followed by the role name after the property name and type. [Figure 4-13](#) shows the initialization of a button with its appropriate values.

You can show that the instance of the owning classifier is related to a particular constructor of a classifier using a dependency line labeled with the keyword `«create»`. You can use any parameters to the constructor when initializing properties simply by using the parameter name. [Figure 4-14](#) shows the application composite diagram tied to a constructor on the Application class.

Figure 4-12. The Application, Window, and Button relationships as a

composite structure

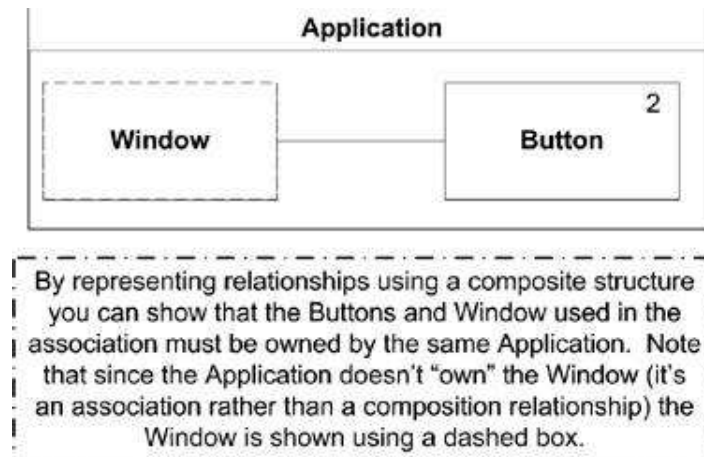


Figure 4-13. Application composite structure with property values

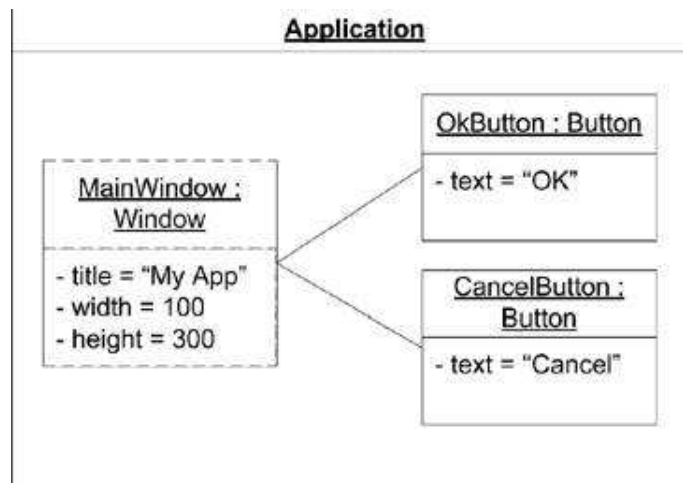
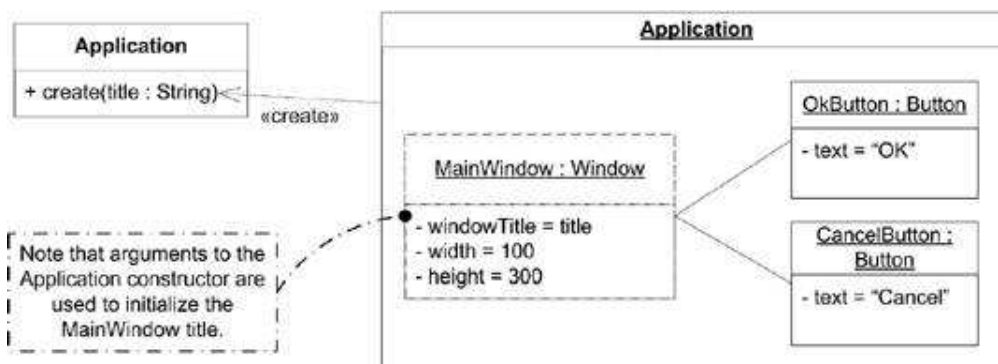


Figure 4-14. Application constructor relating to an instance of an Application



◀ PREV

NEXT ▶

4.2. Collaborations

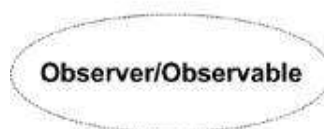
One of the primary purposes for composite structures is to document how a particular piece of functionality is implemented within a system. This organization of elements to realize behavior is called a *collaboration*. A collaboration is a collection of instances wired together using connectors to show the communication flow.

Because the purpose of collaborations is to describe how a particular piece of functionality works, details outside the scope of the desired functionality are typically left off a diagram. Instead, a collaboration diagram shows the required links between instances and the attributes involved in the collaboration. You may have multiple collaborations involving the same instances but showing different views of each based on the functionality expressed. One effective way of showing different views of a classifier is to use *interfaces* to collect related functionality. A single class may realize multiple interfaces, but each collaboration can focus on a single interface. See "[Interfaces](#)" in [Chapter 2](#) for more information.

Within a collaboration, it is helpful to name the instances involved. You typically name an instance based on its role in the collaboration. For example, if you wish to model the Observer/Observable design pattern, you will likely have an instance in the role of Observer and an instance in the role of Subject.

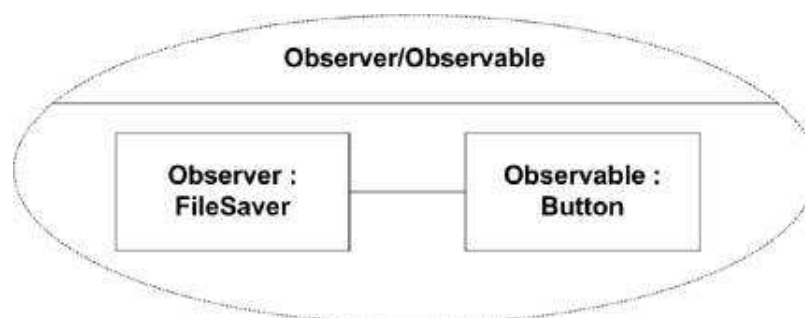
You show a collaboration using a dashed ellipse with the name of the collaboration written inside. [Figure 4-15](#) shows a collaboration named "Observer/Observable."

Figure 4-15. A simple collaboration



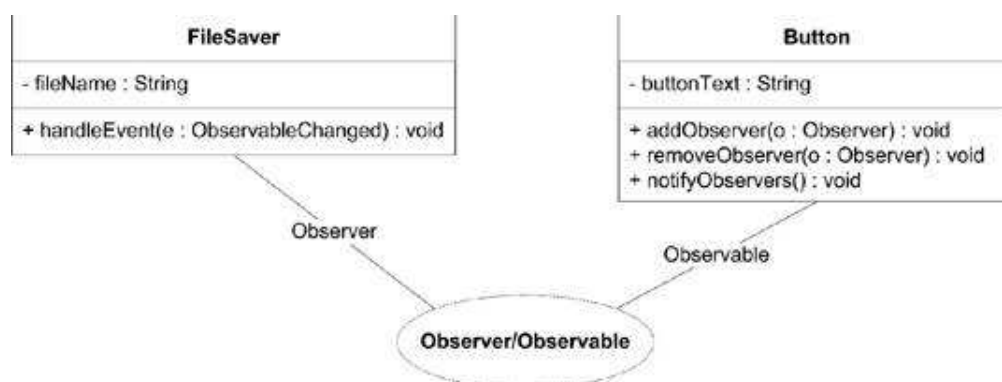
There are two ways to render the details of a collaboration. The first is to add a compartment to the collaboration ellipse and draw the instances involved in the collaboration inside. Links between instances are shown as solid lines. Each instance is named according to its role in the collaboration. [Figure 4-16](#) shows the internal structure of the Observer/Observable collaboration.

Figure 4-16. Internal details of a collaboration



Alternatively, you can show the instances that make up a collaboration outside of the collaboration ellipse and use communication links to tie them back in. In this case, you don't specify the role name inside the classifiers; instead, you write the role name along the communication link for each instance. The advantage of this notation is that you can specify the attributes and operations that are involved in the collaboration (remember, you can leave off any attribute or operation that isn't directly related to the functionality you are expressing). The disadvantage of this notation is that you don't model the direct communication paths between the various instances. [Figure 4-17](#) shows the Observer/Observable pattern using this alternate notation.

Figure 4-17. The Observer/Observable collaboration with details outside of the collaboration ellipse



The UML specification makes it clear that any classifier can be substituted for any member of the collaboration as long as they provide the appropriate attributes and operations to fulfill their role. For example, in the Observer/Observable collaboration shown in [Figure 4-17](#), the Observable must have a means of adding, removing, and notifying observers, and the Observer must have a way to handle changes to the observable. Any other classifier may be used in this collaboration as long as it can provide the message font attribute and the appropriate operation. As mentioned earlier, interfaces lend themselves nicely to collaborations. You can define an Observable interface and programmatically capture exactly what attributes and operations are needed to fulfill a role.

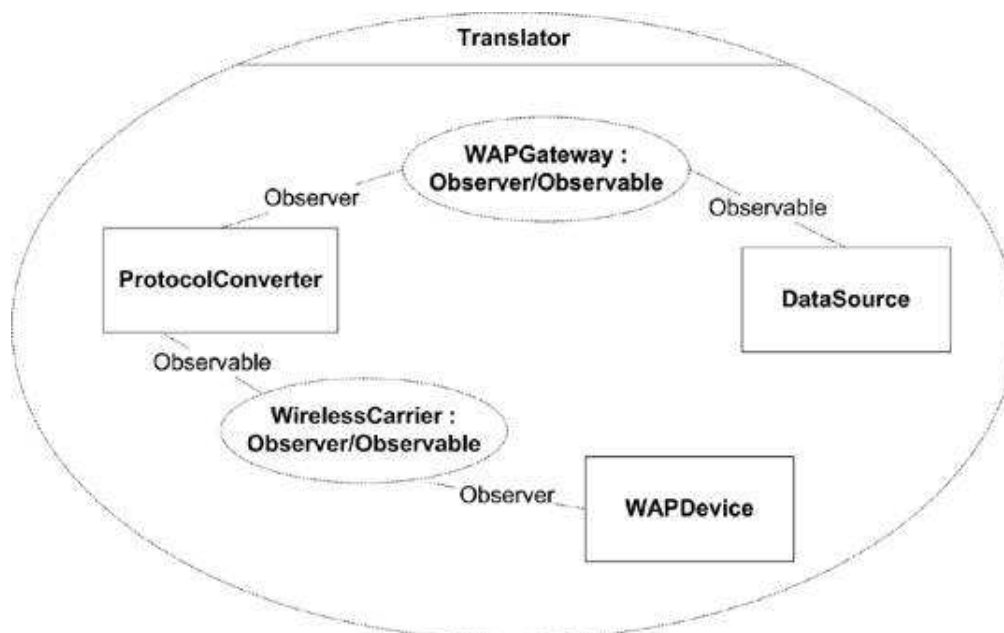
UML 2.0 makes collaborations full classifiers, meaning you can attach sequence diagrams, state machines, or any other behavioral diagram to help capture the details of the implementation.

4.3. Collaboration Occurrences

UML 2.0 introduced a new concept to allow you to attach a collaboration to a specific operation or classifier to show how it is realized by other elements. When you associate a collaboration with an operation or classifier, you create a *collaboration occurrence*. You can think of collaboration occurrences as instances of collaborations. For example, you can use a collaboration occurrence to document how various classes make up a subsystem, what is responsible for persistence, what is really a façade to another subsystem, etc. The advantage of using a collaboration occurrence to document the implementation of functionality is that you can assign role names to internal elements of the classifier. There may be multiple occurrences of a particular collaboration within a classifier, each with different internal elements fulfilling the roles of the collaboration.

You show a collaboration occurrence using the same dashed ellipse used to show a collaboration, except that you list the name of the occurrence followed by a colon, and then the name of the collaboration type. For each role used in the original collaboration, you draw a dashed line from the collaboration occurrence to the element fulfilling the role. You label the dashed line with the name of the role. For example, a collaboration occurrence of our Observer/Observable collaboration is shown in [Figure 4-18](#).

Figure 4-18. Two collaboration occurrences of the Observer/Observable collaboration



Chapter 11. Tagged Values, Stereotypes, and UML Profiles

The UML described in the previous chapters allows you to understand any UML model. You can understand the analysis of a billing system, the implementation model of a CORBA system, a gaming system in C++, or an EJB e-commerce system.

Practitioners rarely work on such diverse systems at one time, however, and no UML model will represent more than one type of application. More commonly, a practitioner (that's you) works with colleagues on one system or a series of closely related systems exclusively. For example, you might design a series of gaming systems, or a series of .Net systems. And, as you might expect, the specific concerns of a gaming system differ profoundly from those of a .Net system.

UML allows toolmakers to create a dialect called a *profile*, tailored to a specific niche. Within a niche, a *stereotype* gives specific roles to elements, and records additional context-specific information in *tagged values*. Profiles also include *constraints* that ensure integrity of use.

A practitioner familiar with a profile will immediately grasp the meaning of a model developed using that profile, because of the unique stereotypes. Moreover, the model contains deeper meaning, because of the tagged values, and the model has a higher degree of integrity, because of the constraints. This gives a distinct advantage to practitioners and tools working in the niche. People and tools unfamiliar with the profile will process it only at a formal level, without any special understanding.

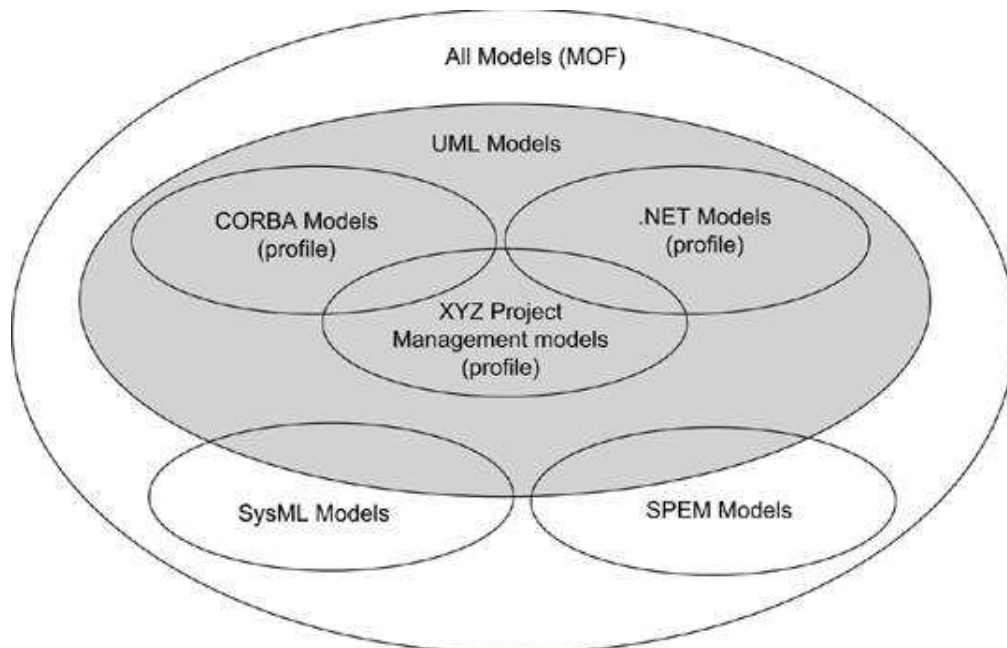
Profiles are *the* standard mechanism to extend UML. The profile mechanism exists within UML so models applying a profile are fully UML compatible. This contrasts sharply with implementors' extensions in loosely specified languages such as C++, in which the specification allows inline assembler and `#pragma` statements, making it virtually impossible to port C++ programs between processors or between different compilers. A UML model applying a profile **is** UML, and any UML tool can process it.

Although a plethora of dialects can fragment the universality of UML, a dialect makes UML more useful. And isn't that what it's all about?

11.1. Modeling and UML in Context

Throughout the evolution of modeling, practitioners, implementors, academics, and other interested parties have found new and innovative ways to model, and new disciplines to model. It soon became apparent that the generality of the canonical UML was not concise enough for practitioners working full time in a particular language, technology, or platform, such as ANSI C++, Struts, or .Net. Moreover, practitioners in similar disciplines, such as process engineering, with different fundamental structures and constraints found UML interesting but not quite appropriate. They can better benefit from a UML-like language other than UML itself. [Figure 11-1](#) illustrates this situation, where the Meta-Object Facility (MOF), explained more fully later in the chapter, comprises all UML models as well as UML-like models.

Figure 11-1. The universe of valid models in UML family



The authors of UML could have specialized UML for common programming languages (such as C++, C#, Java, and Visual Basic) and platforms (such as embedded systems, real-time operating systems (RTOS), EJB, and .NET). This would have created an unworkable modeling language, as each programming language or dialect polluted the specification with conflicting definitions. It would still require an extension mechanism because some "uncommon" language (such as Smalltalk) or new platform/technique or version (like EJB 3.0) would always be missing from the formal specification.

On the other hand, the authors could have stretched UML to greater abstraction to embrace uses other than software development, such as business modeling, modeling the process of software development itself, or modeling systems engineering. This would have made

everything even more abstract. Because designers work in only one specific domain, abstraction impedes concise expression, moving the models further from their domains.

Instead of burdening UML with complexity, or overwhelming it with abstraction, UML's authors factored out everything specific to domains and platforms. They created the profile extension mechanism to address the specific needs of specific application domains. CORBA models, for example, would be concise and precise, but would not influence a .Net model.

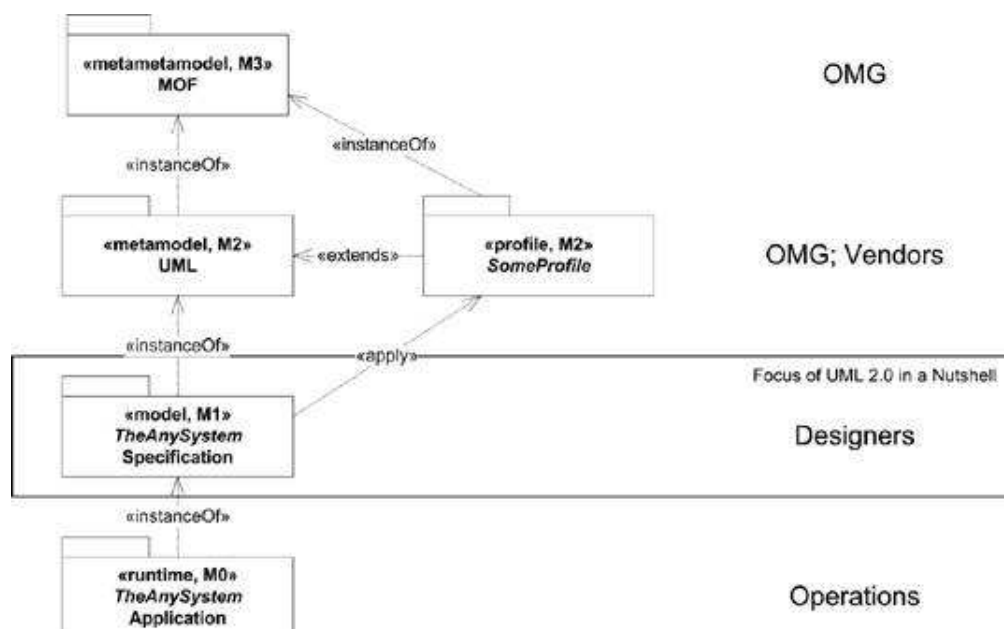
Although UML cousins, such as the Object Management Group's Software Process Engineering Metamodel (SPEM) and SysML.org's Systems Modeling Language (SysML), borrow much structure from UML, they also discard parts that bring no value to their disciplines. SPEM, best known as the basis for the Rational Unified Process (RUP), describes the process of software development in terms of process roles, work products, and activities. Between a software application and the process of software development, the fundamental structure and relationships between the parts change. Although much is similar in the statechart, class, sequence, package, use case, and activity diagrams, for example, there is no implementation or component diagram, or interface. A profile specializing a few elements works, but it must exclude or constrain many basic concepts. MOF factors out the structure of UML itself for reuse in other disciplines. Using MOF, SPEM and SysML become metamodels at the same level as UML. UML remained close to the disciplines of general software development, and business and data modeling. In other disciplines, a new metamodel can be created from adding, dropping, and reorganizing UML packages to allow for a concise modeling of the discipline.

More formally, as seen in [Figure 11-2](#), UML builds on its core infrastructure, MOF. MOF can be used by other modeling languages for other uses. The core UML can be used as is for building models directly, or it can be constrained by one or more profiles.

Each level is a level of abstraction. The levels are named M0, M1, M2, and M3, as they become more abstract. M0 is the concrete systemthe code. M1 is the model of the system (of which M0 is just one realization)the model where designers work. M2 is the language used to describe the model, in this case UML and, optionally, the profiles. M3, MOF, is the language used to describe UML and any similar modeling languages. MOF is far beyond the scope of this book. Suffice it to say, though, that MOF provides a formal infrastructure to UML, which explains why stereotypes resemble classes and components, and why tagged values resemble enumerations and attributes.

The M1 model, specifying your application, may have profiles associated with it. The architect determines the profiles to use according to the platform, language, and tools available. Deciding the profile effectively freezes the M2 layer. As an application modeler, you model classes, attributes, states, and all the other UML elements. You don't define new stereotypes or tagged values. You assign existing stereotypes. You fill in the values appropriate to the tags.

Figure 11-2. Layers of abstraction in UML



Unless you are also building your own tooling for code generation, reporting, and/or tracking, you will employ the profile(s) as is. As a **«singleton»**, for example, the class needs certain supplementary information. The code generator needs the same questions answered for every **«singleton»**; no more, no fewer. It makes no sense to have a **{multithread-safe=true}** tagged value for one **«singleton»** if the code generator doesn't recognize it. If it does recognize it, every **«singleton»** should have it. It really depends on the tool.

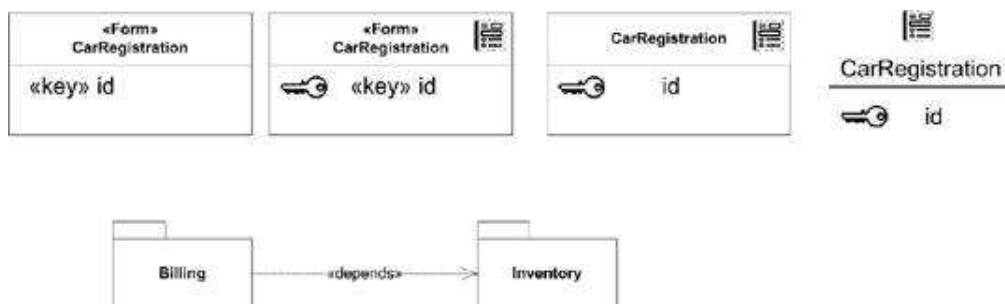
Some teams do build their own tooling. Even then, only the toolsmith works in the M2 layer, and the modelers work in the M1 layer. Changes in the M2 layer must be conservative because one new tagged value in a stereotype can imply revisiting tens or hundreds of elements in the model. Changes in the M2 layer literally change the fundamental meaning of an M1 model. You've been warned.

11.2. Stereotypes

Stereotypes modify the intent of the elements to which they apply. They allow the differentiation of roles of an element within the model. For example, you can quickly differentiate classes stereotyped as *Controller* as having a different role in the system than those stereotyped as *View*.

Visually, UML allows graphical and textual representation of a stereotype. Graphics and text can be combined in various ways for node-type elements, as shown in [Figure 11-3](#). The four elements that you see going across the top of the figure all represent the same combination of *Form* and *key* stereotypes but in different ways. Edge-type elements have only a textual representation, and thus you see `«depends»` on the dependency between *Billing* and *Inventory*.

Figure 11-3. Various representations of stereotypes



When displayed as text, a stereotype is enclosed in guillemots (`«»`), as in `«MyStereotype»`. Because the guillemots require an extended character set to display correctly, you may also use double angle brackets to show a stereotype in 7-bit ASCII, as in `<<MyStereotype>>`.

Graphical icons are neither defined nor standardized by UML. You can expect toolmakers to extend the graphics differently, including coloring or shading, at their discretion. Avoid graphical symbols for interchange of models between different tools. However, within the controlled environment of a compatible set of tools, specialized graphics and/or colors will likely have more visual impact.

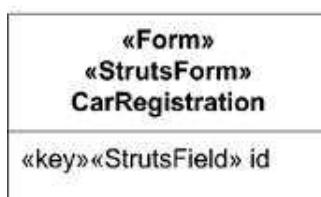


While stereotypes have been around since the initial beta versions of UML, UML 2.0 has introduced significant changes to the 1.x versions:

- Elements may have zero, one, or more than one stereotype. The use and usefulness of stereotypes have become more and more evident. More toolmakers have incorporated them into their products. Modelers found it impossible to use otherwise complimentary tools because an element can have only one stereotype.
- Stereotypes may be nested. A specialized stereotype can build on the structure of a general stereotype. In the example provided later in this chapter, the `EJBPrimaryKey` stereotype extends the `EJBCompField` stereotype, because all primary keys are ordinary fields as well.
- Tagged values are related through a stereotype rather than directly to the element. This avoids the possibility of name clashes with the introduction of multiple stereotypes per element.

You can tag an item in a UML model with more than one stereotype. [Figure 11-4](#) shows multiple stereotypes as a list, with each stereotype enclosed in «guillemots».

Figure 11-4. A class and an attribute, with two stereotypes each

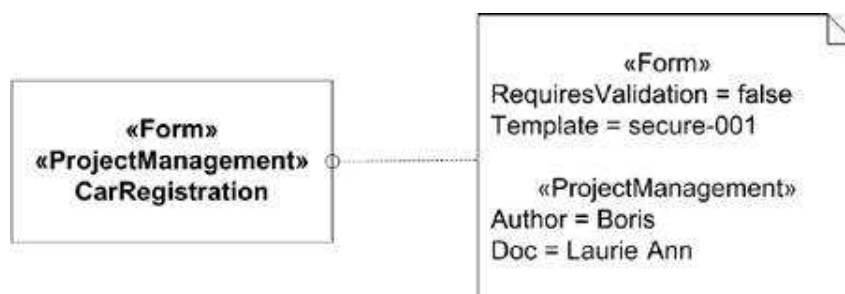


11.3. Tagged Values

Having established an element's role within a system with a stereotype, the element likely needs information not available from the core UML, to fulfill its role. The stereotype defines a number of tagged values. Each tagged value is typed with a datatype number, string, boolean, or user-defined enumeration. The upcoming section "UML Profiles" shows one way in which you might record, or define, the tagged values that you wish to include in a stereotype.

When you show them in a diagram, place the tagged values in a note element that is connected to the declaring element with a dashed line. [Figure 11-5](#) shows the case of multiple stereotypes on one element. To keep the stereotypes and the corresponding tagged values clear, each stereotype is mentioned, and the tagged are values listed separately.

Figure 11-5. Tagged values shown classified by their owning stereotype



At first, you may confuse tagged values with attributes, but they exist at a different level of abstraction. Attributes, defined in the design model (M1), exist in the runtime system (M0). Tagged values, defined in the profile (M2), exist only in the design model (M1). The tagged values may provide hints to help the generation of code, either by human or machine. A tagged value of `{optimize=space}` will probably affect the code ultimately, although the actual value itself never appears in the code.

[◀ PREV](#)[NEXT ▶](#)

11.4. Constraints

Stereotypes give roles to elements. Tagged values provide role-specific information enriching the element in its role. While atomically the element knows its role and has all the information to fulfill it, the element must still interact with its neighbors. The element and their roles must be in harmony with the architectural vision. The element must also be internally consistent. Constraints (see "[Constraints](#)" in [Chapter 2](#)) provide the mechanism to specify rules for correct usage of the stereotyped elements.

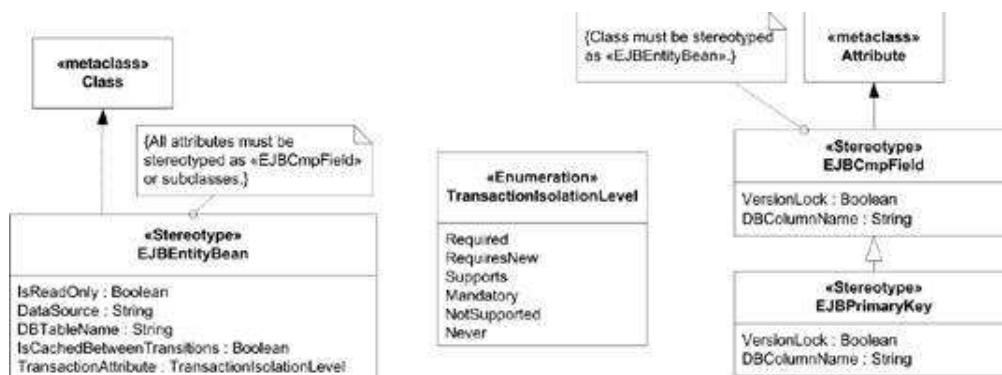
[◀ PREV](#)[NEXT ▶](#)

11.5. UML Profiles

UML profiles combine the concepts of stereotypes, tagged values, and constraints to provide a coherent and concise dialect of UML for a specific family of applications. To make much use of a profile, some tooling must be provided. The application model drives code or application generation, so you have little or no control over the stereotypes, tagged values, or constraints comprising the profile. This section discusses the use of existing profiles (as opposed to defining your own).

[Figure 11-6](#) depicts a partial UML profile defining a stereotype with its associated tagged values and a couple of constraints, as you might receive in a vendor's documentation. The profile extends classes with a stereotyped class, `«EJBEntityBean»`. It extends attributes with two stereotyped attributes: `«EJBPrimaryKey»` and `«EJBCompField»`. It declares the respective tagged values for the stereotyped classes and attributes, and it declares the enumeration, `transactionIsolationLevel`, to define the allowable values for the `TransactionAttribute` tagged value. The profile also adds the constraints that `«EJBEntityBean»` classes must have attributes of type `«EJBCompField»` and `«EJBPrimaryKey»`. Furthermore, attributes having these stereotypes can exist only in an `«EJBEntityBean»` class. From your point of view, the profile, along with its constituent stereotypes and tagged values, is read-only because it tells what the third-party tool expects in order to do its job.

Figure 11-6. A partial specification UML profile



[Figure 11-7](#) shows a portion of a model using the profile declared in [Figure 11-6](#). [Figure 11-8](#) indicates how the tagged value structures in the model relate back to the profile declaration. The notes containing the tagged values make the notation bulky if you need to show a set of tagged values for every class, attribute, operation, and relationship.

When aiming to expose the structure and relationships of a system, class diagrams rarely show tagged values. Referring again to [Figure 11-7](#), one tiny class having two attributes becomes a constellation of four diagram elements. A modest class diagram of 10 classes can easily require 100 extra notes to show the tagged values, hopelessly confusing the diagram and destroying its impact. Instead, modelers often maintain the tagged values invisibly in the model, where tools can extract them as needed.

Figure 11-7. A stereotyped class and its stereotyped attributes, with their respective tagged values

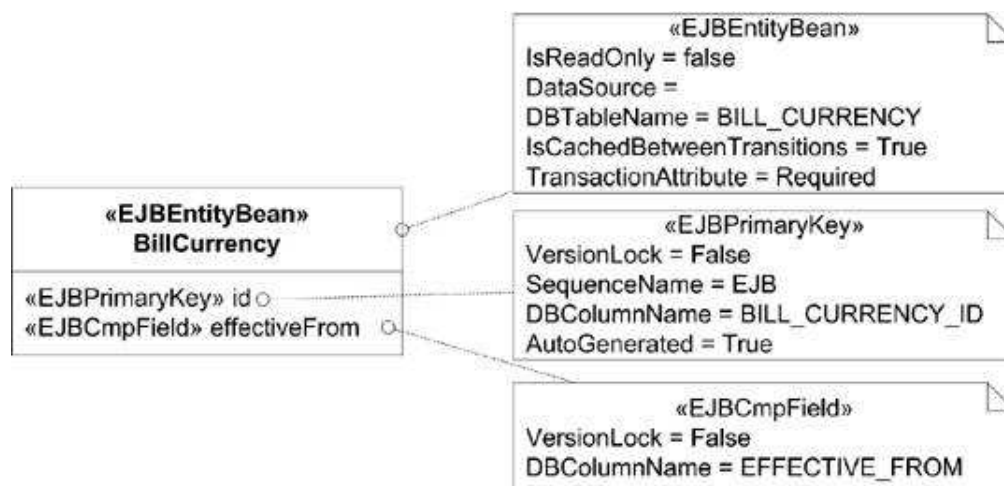
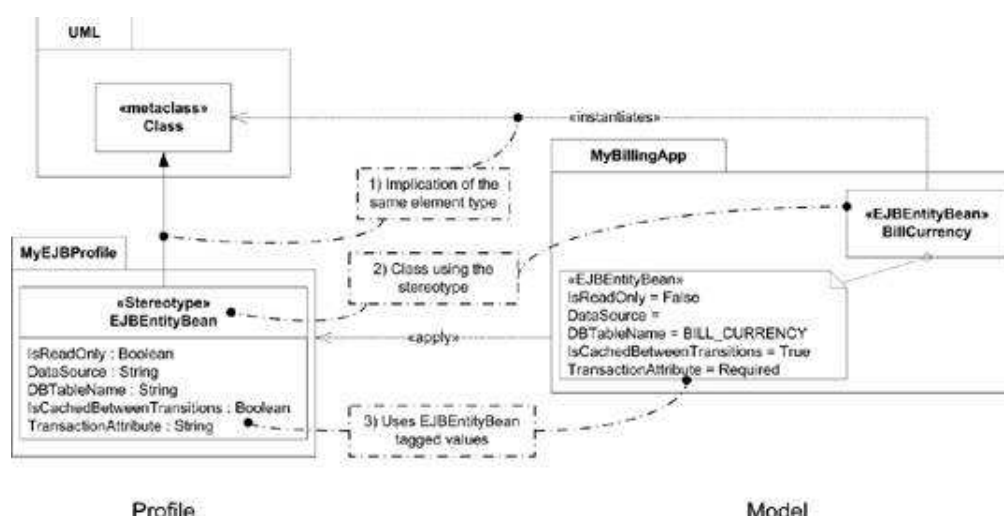


Figure 11-8. The partial specification UML profile, showing model elements applying it



The UML from the profile and the model is shown in [Figure 11-8](#). This illustrates the relationships between the declarations of the profile and a conforming model. The stereotypes in the profile extend the concept of an element. In this case, the stereotype **«EJBEntityBean»** extends ordinary classes, as shown (1) by the implication of the same element type (metaclass), **Class**. In the models applying this profile, only classes may have that stereotype, as shown by the relationship (2). Having established that the **BillCurrency** is an **«EJBEntityBean»**, it now has all the **«EJBEntityBean»** tagged values, as shown by the relationship (3). Stereotypes that extend other stereotypes, as **«EJBPrimaryKey»** extends **«EJBCompField»**, have all the parent's tagged values. Ideally, tooling will aid, enforce, and validate by making available the correct tagged values and defaults according to the stereotype's definition. If not, the modeler must rely on discipline.

◀ PREV

NEXT ▶

11.6. Tools and How They Use Profiles

UML tools use profiles to provide a spectrum of solutions. Tools providing Model-Driven Architecture (MDA) solutions have transformations from the Platform-Independent Model (PIM) to Platform-Specific Model (PSM) and from the PSM to the application. See [Appendix A](#) for a fuller discussion of MDA.

The OMG conceived the MDA as a vision rather than a specified method. Each vendor has a different, sometimes radically different, approach to MDA. Consequently, although the concepts of the PIM and the PSM vary greatly from one vendor to another, any one vendor's concept of the PIM and the PSM is strict and concrete. The extra roles and information, introduced as the PIM is refined to the PSM and the PSM to code, must be well defined and well controlled. Profiles provide the definition of the information to be captured and the constraints on a valid model. Each tool validates conformity to a profile in its own way. The PIM needs only one profile because it can be used and reused for different platforms. Each PSM, on the other hand, potentially needs a different profile because each specific platform has different issues and transformations to arrive at optimal code.

Tools previous to MDA still use profiles. In general, they provide a model-to-code code generation feature and often a code-to-model reverse engineering feature. To faithfully generate runnable code, many details must be stored in the model. Many language-dependent features can't be recorded in core UML. For example, UML doesn't recognize the Java keyword `strictfp`, which indicates floating-point processing. Without tagged values, a reverse-engineered system would not faithfully reproduce the same code when forward engineered.

[◀ PREV](#)[NEXT ▶](#)

Chapter 12. Effective Diagramming

This chapter illustrates how you can create effective class diagrams. The diagrams presented in the reference section of this book have few elements. Those diagrams illustrate individual diagram elements, clear of any clutter; they aren't meant to illustrate diagram types. The diagrams in the reference section don't represent systems that you will encounter in real life. This final chapter illustrates some techniques you can use to make real-life diagrams convey your thoughts more effectively.

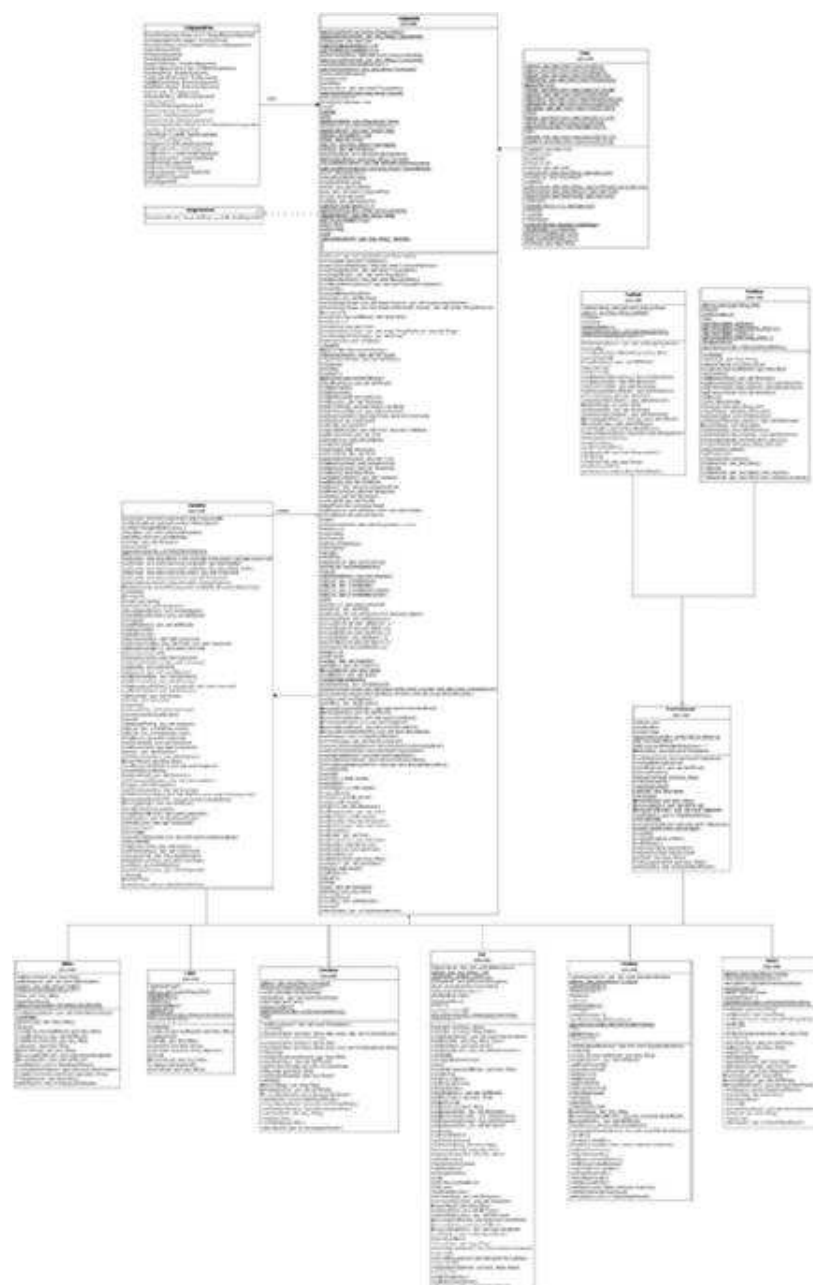
[◀ PREV](#)[NEXT ▶](#)

12.1. Wallpaper Diagrams

The classic novice class diagram displays all classes, attributes, operations, relationships, and dependencies for a system. As the number of classes grows, the diagram becomes huge and unmanageable. Imprudent reverse engineering drags in clutter. [Figure 12-1](#) diagrams just a very small part of the Java Abstract Windowing Toolkit (AWT). You will not be able to read the diagram because it normally prints as an unwieldy nine pages. You're paying for the paper, so the diagram is shown here in a greatly condensed form.

Effective writers convey ideas through language constructs: sentences and paragraphs organize text, details reinforce ideas, and true but irrelevant facts are omitted. Modelers desiring to convey an idea be it a structural overview, a pattern implementation, a detailed subsystem, or the detailed context of one class must also use economy and focus. [Figure 12-1](#) has neither economy nor focus. The diagram throws a mass of insignificant detail at readers, leaving the burden of understanding on them. It is a false economy to think that one diagram can present all the information of a multidimensional model.

Figure 12-1. An overloaded and ineffective class diagram



12.1.1. Modeling Versus Diagramming

Before making [Figure 12-1](#) more effective, let's distinguish modeling from diagramming. The act of modeling furthers your understanding of the system under study. You use diagrams to elaborate the model. You confirm the model integrity with new views. You organize your understanding. You extend your knowledge. You challenge your understanding. By seeing the known, you can infer, dictate, or investigate the yet to be known. [Figure 12-1](#) might be seen during modeling, but only as an intermediate step; such diagrams don't demonstrate UML's true power to communicate.

Diagramming, on the other hand, expresses understanding for the benefit of others. You may

be a domain expert by background, or you may have recently synthesized expertise from modeling with domain experts. You have worked hard to understand, and to put your understanding into a model, perhaps over weeks or months; now you must make that understanding immediately accessible to your readers. Your model should be concise so that they understand. If there is to be contention, let it be about real ideas, not ambiguity.

Diagramming reveals to others what you discover through modeling .

It doesn't matter whether you formally create a model in a modeling tool, whether the model is implicitly in your head, or whether the model resides in another form: each diagram drawn from that model should express one or a very limited number of related concepts. You choose the idea to express, and you express it from the elements in the model. Presenting your ideas in UML requires multiple diagrams, just as writing a section in this chapter requires multiple paragraphs.

Implicitly, you may think that discussions of layout and effective diagrams relate only to human-drawn diagrams. Automated utilities create one-way transformations to UML of complex text or XML-based configuration files or even nontext systems. This visual UML presents an alternative and complementary view to the difficult-to-understand source text, especially when used as overviews and sanity checks. Most such utilities automatically lay out the resulting diagrams without allowing the user to arrange the diagrams. (Manual arrangement would impose undue work on the user.) The need for clarity and focus applies equally to UML produced by tools because the reader is still the same.

Though UML provides an excellent vehicle for both modeling and diagramming, the resultant diagrams aren't necessarily the same. Modeling tolerates larger diagrams with a muddled focus. A large diagram makes a better playground because the cost of refactoring many small diagrams outweighs the benefits of the clarity those diagrams provide, at least in the early stages of design. Once you understand the model and are comfortable with it, you can fission the resultant model more easily into packages and concise diagrams. Ease during this phase validates the model's completeness; pain in diagramming indicates incompleteness in modeling. Details forgotten against the vastness of the model stand out when the focus narrows to a very small context.

Your documentation, UML and otherwise, must answer when you are absent. When you're publishing a guide to a toolkit, framework, or system, your audience will fail to exploit your offering without clear and focused diagrams. At the same time, once the model is clearly understood and concisely presented, discard the playground diagrams to avoid confusion.

As with any other endeavor, shortcuts can be appropriate. If only you or an application generator access a model, you don't need to publish it for others. If the model is implicitly valid or you have better validation mechanisms, such as model checkers, you need not bother creating concise diagrams. A code generator, for example, doesn't care about understandable diagrams.

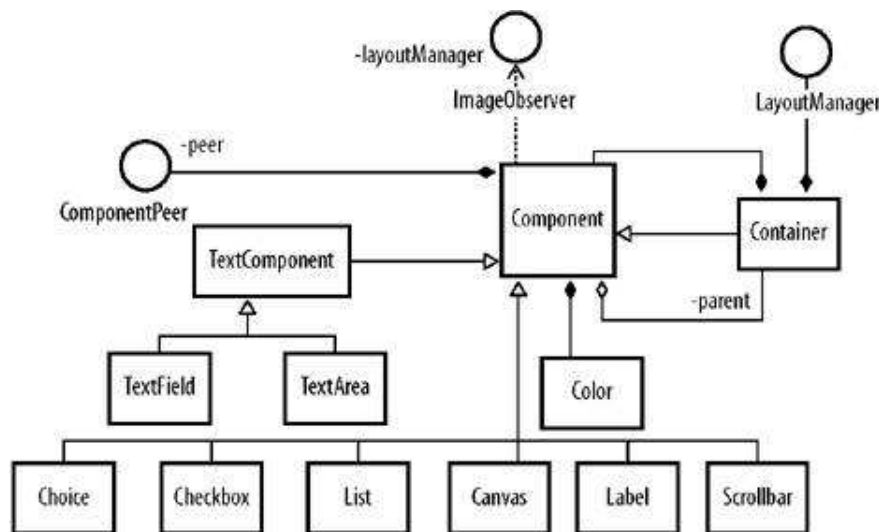
No one diagram is definitive in all aspects. To be effective, a diagram must have unity of purpose. As a 2D view of the model, a diagram necessarily leaves out details in order to emphasize important features. The more economic the details, the more salient the features appear in relief. Several diagrams, each with a different purpose, provide a clearer understanding than one diagram that has all the details.

When diagramming, consider the purpose. The following sections detail several errors that compromise the effectiveness of UML.

12.1.2. Structure and Interrelationships Among Classes

[Figure 12-1](#) shows a class diagram cluttered by details. It gives no insight. [Figure 12-2](#) shows

Figure 12-2. The structure of commonly used components within the Java AWT



12.1.3. Separate Inheritance and Class Interrelationships

Figure 12-3. Structure of Components and Containers in the AWT

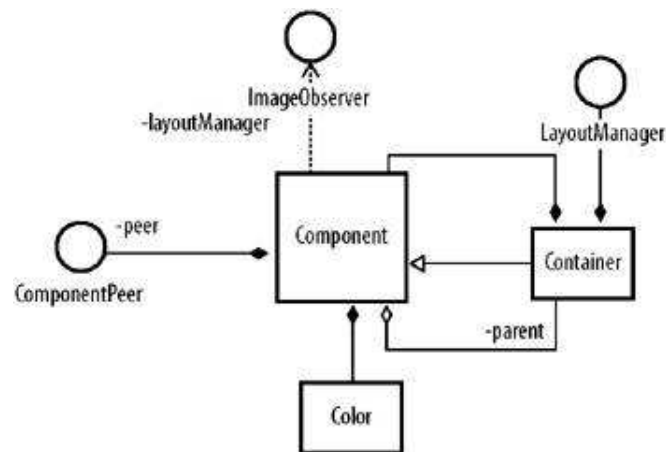
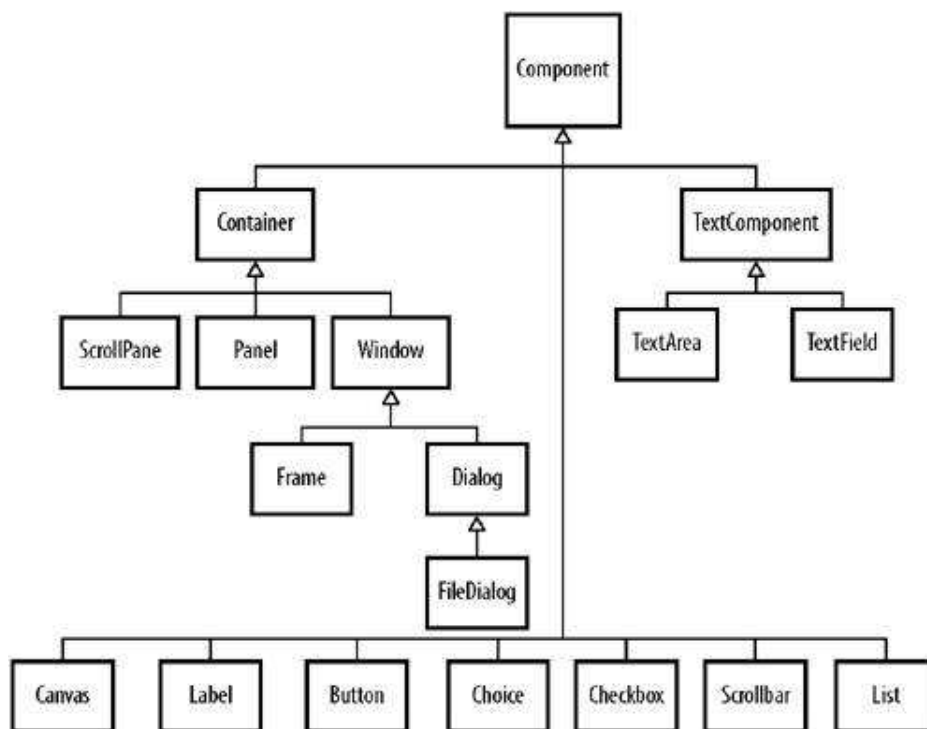


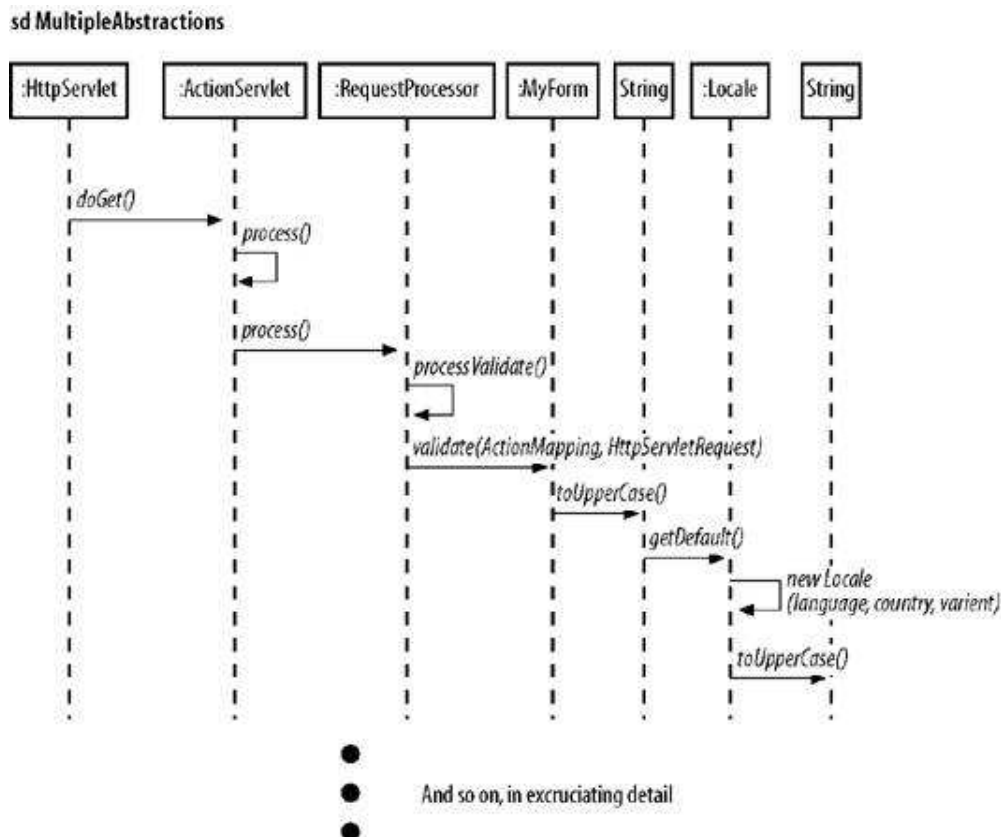
Figure 12-4. Inheritance tree of Component in the AWT



12.2. Sprawling Scope

[Figure 12-5](#) shows an example of a poorly focused sequence diagram. Sequence diagrams risk taking on too many interactions at once. This one shows the Struts interactions, the application interactions, and then the detailed workings of the Java libraries.

Figure 12-5. Sequence diagram of HTTP form processing, spanning Struts, application, and java.lang

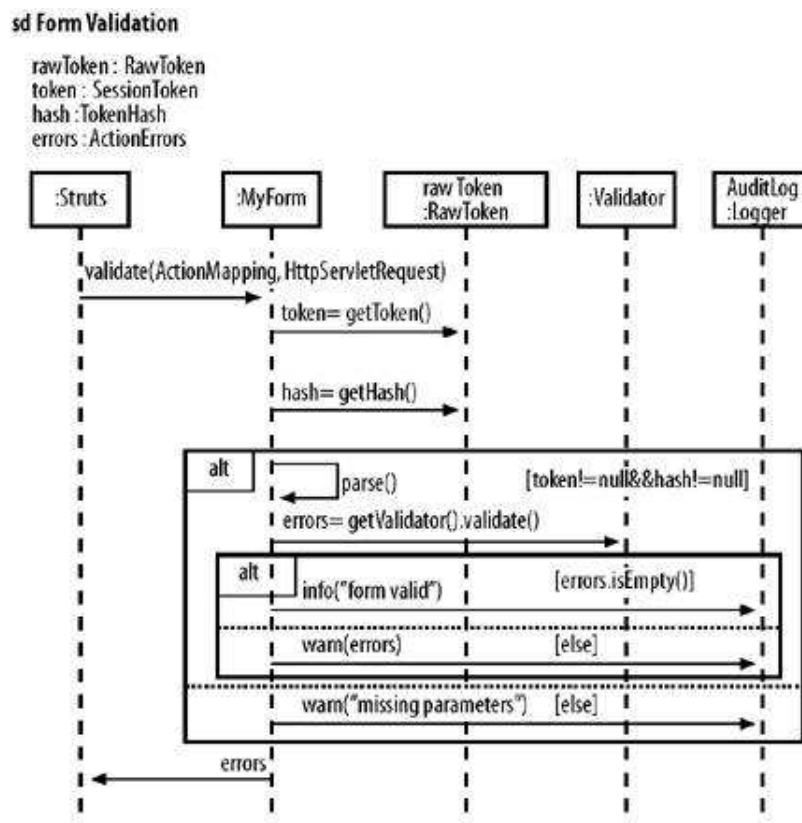


Elements at the high and low end of this diagram are clearly out of scope of the application. One rarely cares how Struts, as an off-the-shelf framework, works. It calls the application logic at the appropriate time. Similarly, just how the internals of the standard libraries carry out their obligations normally doesn't matter in terms of understanding the application. Both the structure of HTTP request processing and the workings of `Strings` are God-given as far as the application developer is concerned. However, by showing a few elements that are out of scope, the diagram allows you to see the context of the application within the greater framework, and that's often useful when selecting an architecture. Similarly, drilling down into the base libraries exposes thread or performance issues. These are exceptional needs; there is no call to complicate the description of an application with these exploratory needs. From the readers' point of view, nonapplication elements just cloud understanding of the business

logic, the only thing over which the developer exerts control.

[Figure 12-6](#) shows the processing for another HTML form. It gets to the point. It is concise enough for one page. And it focuses on interactions between the application classes; the interactions are under control of the development team.

Figure 12-6. Sequence diagram with scope restricted to programmer-defined classes

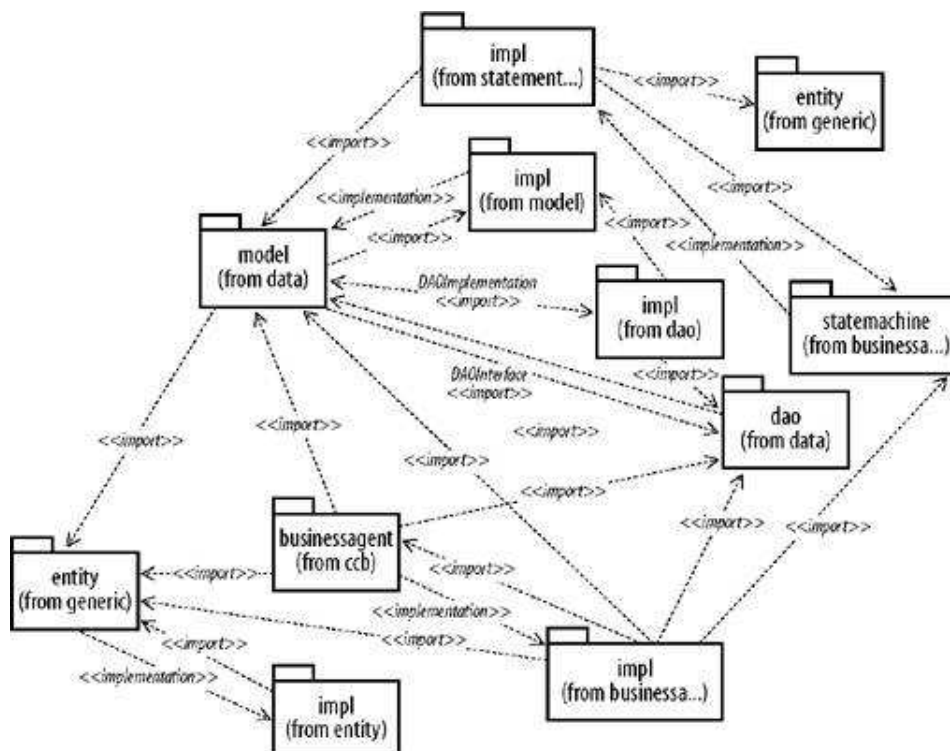


12.3. One Diagram/One Abstraction

Earlier, in [Figures 12-2](#) and [12-3](#), we separated structure from inheritance. Each diagram contains fewer elements than the all-encompassing diagram shown in [Figure 12-1](#), or even the more modest one shown in [Figure 12-2](#), so they are easier to understand. [Figure 12-7](#) shows package dependencies. The diagram records both the import graph and code generation directives. The superimposition of these two roles results in a diagram with pairs of opposing dependencies between elements. Don't worry if you don't quite understand its meaning; we'll clean it up in a moment.

[Figure 12-7](#) fails as a diagram for several reasons. Most strikingly, you don't know where to start: the diagram has no center of gravity. This leads to a lack of flow or direction; do you read the diagram left to right, top to bottom, or inside out? The apparently cyclic dependencies confound any sense of direction. Although opaque to the initiated, the problem with [Figure 12-7](#) stems from trying to use one diagram for two related but different ends: controlling imports and directing code generation.

Figure 12-7. Package diagram controlling imports and code generation directives



[Figure 12-8](#) addresses the issue of imports only. Elements higher in the diagram depend on elements below. No cycles exist. You can absorb the meaning of the diagram quickly.

Removing the imports from [Figure 12-7](#) simplifies it more than you might imagine. Instead of forming a graph like the imports, the directives, as shown in [Figure 12-9](#), just link separate elements with target packages. The noise of the import graph complicated a simple picture. The complicated, large diagram becomes a small diagram of four separate sections.

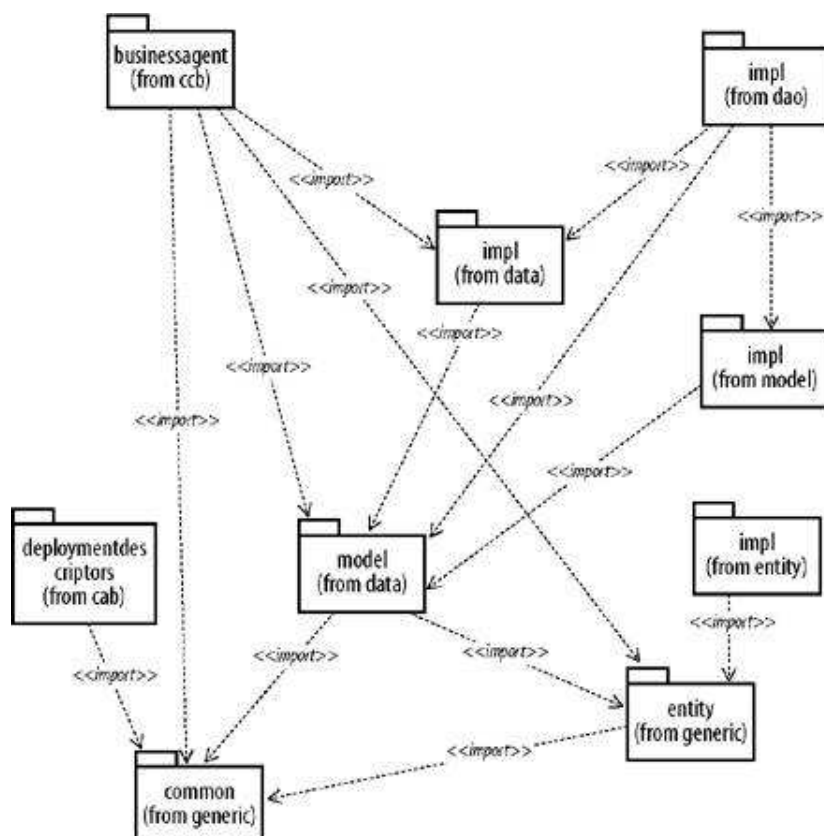
[◀ PREV](#)[NEXT ▶](#)

12.4. Besides UML

UML provides description that's difficult to match with plain text. Its formalism allows a few lines and boxes to unequivocally and immediately convey what would require time to assimilate from text. The text replaced by diagrams would be, at best, dull and legalistic, and at worst, vague, ambiguous, and open to interpretation. Either way, well-written UML is more effective.

While it may be true that a picture is worth a thousand words, a picture with words is worth much more than just a picture. Perhaps in the context of MDA and executable models, UML can provide a minimum specification for an application generator. In most cases, complete system specification or system documentation requires commentary to explain, emphasize, and give nuance to the diagrams. Diagrams alone risk burying or glossing over details as much as text alone. Diagrams and text reinforce each other. You can describe the context and

Figure 12-8. Package imports

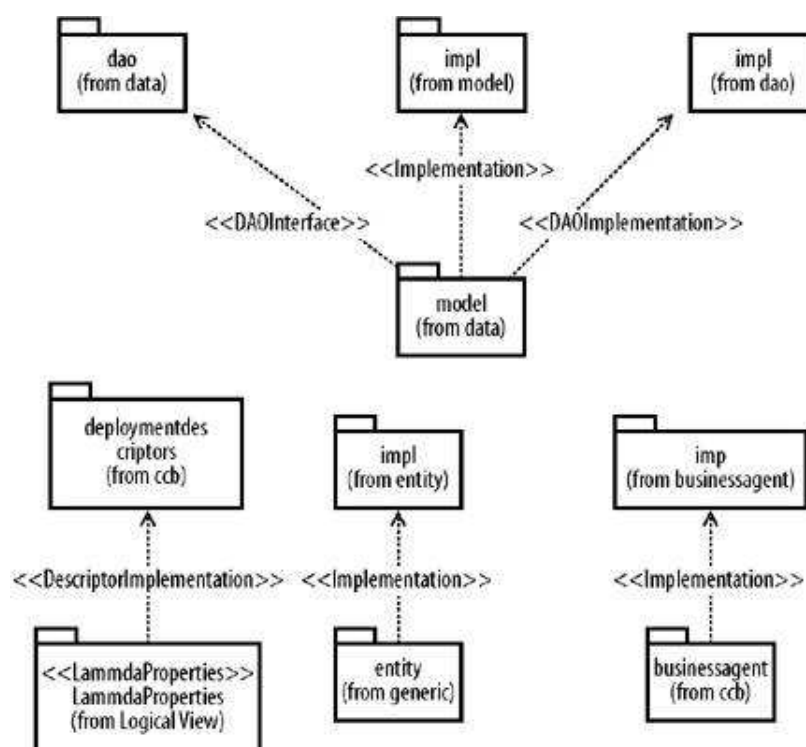


major points with text, then drive home those points with a good diagram and the formalism of UML.

As seen throughout this chapter, you can't effectively document classes with large APIs on a single, class diagram. Hundreds of attributes and operations become insignificant on a diagram. The operation and attribute sections of a class box don't allow for descriptions. Tools such as Doxygen (<http://www.doxygen.org>) or Javadoc extract the most current source comments into searchable web pages. Though the details they present are likely to change and therefore will make diagrams obsolete, when they are made part of the software build process, they remain up to date with the latest software changes.

Remember that diagramming requires neither model nor modeling tool. UML practitioners often simply draw diagrams by hand or with simple drawing tools. Modeling tools are expensive acquisitions, in terms of time and distraction, if not in money: open source or proprietary, UML tools take time to master. Whiteboards, notepads, and simple drawing tools deliver much project value with no learning curve. Use them until you are comfortable with UML and its place in the project. With a firm understanding of UML and your process, choosing a dedicated UML tool and changing your process to incorporate it becomes less risky.

Figure 12-9. Code generation directives



[◀ PREV](#)[NEXT ▶](#)

Appendix B. The Object Constraint Language

The Object Constraint Language 2.0 (OCL) is an addition to the UML 2.0 specification that provides you with a way to express constraints and logic on your models. For example, you can use OCL to convey that a person's age must always be greater than 0 or that a branch office must always have one secretary for every 10 employees.

OCL isn't new to UML 2.0; it was first introduced in UML 1.4. However, as of UML 2.0, it was formalized using the Meta-Object Facility and UML 2.0. From a user's perspective the language has been updated and refined but the fundamentals remain the same. This appendix introduces the basic concepts of OCL. For more detailed information, consult the OCL specification available from the Object Management Group's web site (<http://www.omg.org/>).

[◀ PREV](#)[NEXT ▶](#)

B.1. OCL Basics

The Object Constraint Language is just that: a language. It obeys a syntax and has keywords. However, unlike other languages, it can't be used to express program logic or flow control. By design, OCL is a *query-only* language; it can't modify the model (or executing system) in any way. It can be used to express preconditions, postconditions, invariants (things that must always be `TRUE`), guard conditions, and results of method calls.

OCL can be used virtually anywhere in UML and is typically associated with a classifier using a note. When an OCL expression is evaluated, it is considered to be instantaneous, meaning the associated classifier can't change state during the evaluation of an expression.

B.1.1. Basic Types

OCL has several built-in types that can be used in OCL expressions:

Boolean

Must be either `true` or `false`. Supports the logical operators `and`, `or`, `xor`, `not`, `implies`, and `if-then-else`.

Integer

Any integer value (e.g., 100, -12345, 5211976, etc.). Supports the operators `*`, `+`, `-`, `/`, and `abs( )`.

Real

Any decimal value (e.g., 2.222003, -67.76, etc.). Supports the operators `*`, `+`, `-`, `/`, and `floor( )`.

String

A series of letters, numbers, or symbols interpreted as a string (e.g., "All writing and no play make Dan..."). Supports the operators `concat( )`, `size( )`, and `substring( )`.

In addition to the built-in types, any classifier used in your UML model is recognized as a type by OCL. Because OCL is a strongly typed language, you can't compare values of one type directly with values of another type.

B.1.2. Casting

OCL does support *casting* objects from one type to another as long as they are related through a generalization relationship. To cast from one type to another use the operation

`oldType.oclAsType(newType)` . For example, to cast a Java `List` to an `ArrayList` to call the `size()` operation, use the expression:

```
List.oclAsType(ArrayList).size()
```

If the actual object isn't an instance of the new type, the expression is considered undefined.

[< PREVIOUS](#)[NEXT >](#)

B.2. OCL Syntax

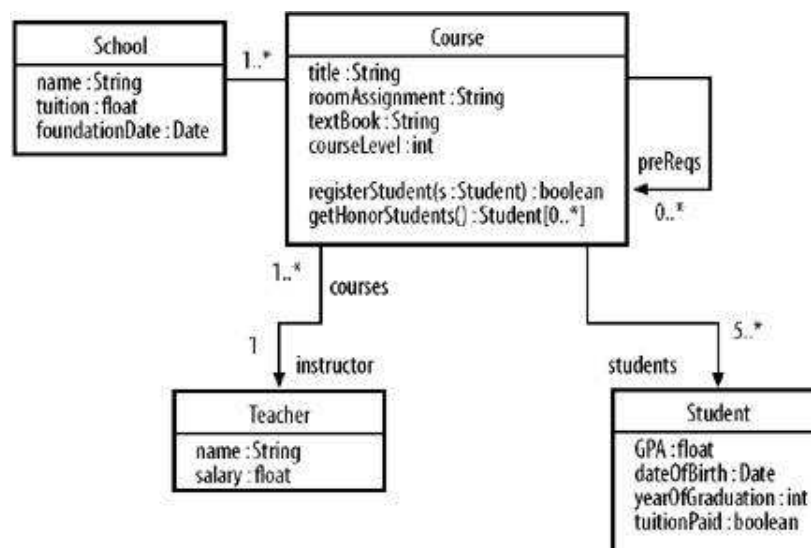
The remainder of this chapter uses examples from the class diagram shown in [Figure B-1](#).

B.2.1. Constraints on Classifiers

Each OCL expression must have some sense of *context* that an expression relates to. Often the context can be determined by where the expression is written. For example, you can link a constraint to an element using a note. You can refer to an instance of the context classifier using the keyword `self`. For example, if you had a constraint on `Student` that their GPA must always be higher than 2.0, you can attach an OCL expression to `Student` using a note and refer to the GPA as follows:

```
self.GPA > 2.0
```

Figure B-1. Example class diagram used in this chapter



It's important to realize that this OCL expression is an invariant, meaning the system would be in an invalid state if a student's GPA dropped to less than 2.0. If you want to allow a GPA of less than 2.0 and send out a letter to the student's parents in the event such a low GPA is achieved, you would model such behavior using a UML diagram such as an activity or interaction diagram.

You can follow associations between classifiers using the association end names as though they were attributes of the originating classifier. The following invariant on `Course` ensures that the instructor is being paid:

```
self.instructor.salary > 0.00
```

If an association has a multiplicity of 0..1, you can treat the association end as a `Set` and check to see if the value is set by using the built-in `notEmpty()` operation. To call the `notEmpty()` operation on a set you must use an arrow (`->`) rather than a dot (`.`). See "[Collections](#)" for more information on sets. The following invariant on `Course` enforces that a course has an instructor:

```
self.instructor->notEmpty()
```

If an association role name isn't specified, you can use the classifier name. The following invariant on `School` checks that each course has a room assignment:

```
self.Course->forAll(roomAssignment <> 'No room!')
```

Comments can be inserted into an expression by prefixing each comment with two dashes (`--`), like this:

```
-- make sure this student could graduate
self.GPA > 2.0
```

If you can't determine the context of an expression by looking at the UML model, or if you want to be explicit with the context, use the OCL keyword `context`, followed by the classifier name. If you use this notation, you should say what the expression represents. In the following case, we're showing an invariant, so we use the keyword `inv`. Other types of expressions are explained in later sections.

```
context Student
inv: self.GPA > 2.0
```

Instead of using the keyword `self`, you can assign a name to a classifier that you can use in the expression. Write the name you want to use, followed by a colon (`:`) and then the classifier name. For example, you can name the instance of `Student` as `s`:

```
context s : Student
inv: s.GPA > 2.0
```

Finally, you can name an expression by placing a label after the expression type but before the colon (`:`). The label is purely decorative; it serves no OCL function.

```
context s : Student
inv minimumGPARule: s.GPA > 2.0
```

B.2.2. Constraints on Operations

Beyond basic classifiers, OCL expressions can be associated with operations to capture

preconditions and postconditions. Place the signature of the target operation (classifier, operation name, parameters, etc.) after the `context` keyword. Instead of the invariant keyword, use either `pre` or `post` for preconditions and postconditions, respectively.

The following expression ensures that any student who will be registered for a course has paid their tuition:

```
context Course::registerStudent(s : Student) : boolean
pre: s.tuitionPaid = true
```

When writing postconditions, you can use the keyword `result` to refer to the value returned by an operation. The following expressions verify that a student's tuition was paid before registering for a course and that the operation `registerStudent` returned `true`:

```
context Course::registerStudent(s : Student) : boolean
pre: s.tuitionPaid = true
post: result = true
```

As you can with invariants, you can name preconditions and postconditions by placing a label after the `pre` or `post` keywords:

```
context Course::registerStudent(s : Student) : boolean
pre hasPaidTuition: s.tuitionPaid = true
post studentWasRegistered: result = true
```

Postconditions can use the `@pre` keyword to refer to the value of some element *before* an operation executes. The following expression ensures that a student was registered and that the number of students in the course has increased by 1. This expression uses the `self` keyword to reference the object that owns the `registerStudent` operation.

```
context Course::registerStudent(s : Student) : boolean
pre: s.tuitionPaid = true
post: result = true AND self.students = self.students@pre + 1
```

You may specify the results of a *query* operation using the keyword `body`. Because OCL doesn't have syntax for program flow, you are limited to relatively simple expressions. The following expression indicates that honors students are students with GPAs higher than 3.5. The collection syntax used in this example is described in the "[Collections](#)" section.

```
context Course::getHonorsStudents( ) : Student
body: self.students->select(GPA > 3.5)
```

B.2.3. Constraints on Attributes

OCL expressions can specify the initial and subsequent values for attributes of classifiers. When using OCL expressions with attributes, you state the context of an expression using the classifier name, two colons (`::`), the attribute name, another colon (`:`), and then the type of the attribute. You specify the initial value of an attribute using the keyword `init`:

```
context School::tuition : float
init: 2500.00
```

You can specify the value of attributes after their initial value using the `derive` keyword. The following example increases the tuition value by 10% every time you query it:

```
context: School::tuition : float
derive: tuition * 10%
```

[< PREVIOUS](#)[NEXT >](#)

B.3. Advanced OCL Modeling

Like any other language, OCL has an order of precedence for operators, variable declarations, and logical constructs (only for evaluating your expressions, not for program flow). The following sections describe constructs that you can use in any OCL expression.

B.3.1. Conditionals

OCL supports basic boolean expression evaluation using the `if-then-else-endif` keywords. The conditions are used only to determine which expression is evaluated; they can't be used to influence the underlying system or to affect program flow. The following invariant enforces that student's year of graduation is valid only if she has paid her tuition:

```
context Student inv:
  if tuitionPaid = true then
    yearOfGraduation = 2005
  else
    yearOfGraduation = 0000
  endif
```

OCL's logic rules are slightly different from typical programming language logic rules. The boolean evaluation rules are:

1. `True` OR-ed with anything is `true`.
2. `False` AND-ed with anything is `false`.
3. `False` IMPLIES *anything* is `True`.

The `implies` keyword evaluates the first half of an expression, and, if that first half is `true`, the result is taken from the second half. For example, the following expression enforces that if a student's GPA is less than 1.0, their year of graduation is set to 0. If the GPA is higher than 1.0, Rule #3 applies, and the entire expression is evaluated as `true` (meaning the invariant is valid).

```
context Student inv:
  self.GPA < 1.0 IMPLIES self.yearOfGraduation = 0000
```

OCL's boolean expressions are valid *regardless of the order of the arguments*. Specifically, if the first argument of an `AND` operator is undefined, but the second operator is `false`, the entire expression is `false`. Likewise, even if one of the arguments to an `OR` operator is undefined, if the other is `true`, the expression is `true`. `If-then-else-endif` constructs are evaluated similarly; if the chosen branch can be evaluated to `True` or `false`, the nonchosen branch is completely disregarded (even if it would be undefined).

B.3.2. Variable Declaration

OCL supports several complex constructs you can use to make your constraints more expressive and easier to write. You can break complex expressions into reusable pieces (within the same expression) by using the `let` and `in` keywords to declare a variable. You declare a variable by giving it a name, followed by a colon (:), its type, an expression for its value, and the `in` keyword. The following example declares an expression that ensures a teacher of a high-level course has appropriate salary:

```
context Course inv:
  let salary : float = self.instructor.salary in
  if self.courseLevel > 4000 then
    salary > 80000.00
  else
    salary < 80000.00
  endif
```

You can define variables that can be used in multiple expressions on a classifier-by-classifier basis using the `def` keyword. For example, instead of declaring `salary` as a variable using the `let` keyword, you can define it using the `def` keyword for the `Course` context. Once you define a variable using the `def` keyword, you may use that variable in any expression that is in the same context. The syntax for the `def` keyword is the same as that for the `let` keyword:

```
context Course
def: salary : float = self.instructor.salary
```

So, now you can write the previous invariant as:

```
context Course inv:
  if self.courseLevel > 4000 then
    salary > 80000.00
  getHonorsStudents else
    salary < 80000.00
  endif
```

B.3.3. Operator Precedence

OCL operators have the following order of precedence (from highest to lowest):

- `@pre`
- dot (.) and arrow (->) operations
- `not` and unary minus (-)
- `*` and `/`
- `+` and `-`
- `if-then-else-endif`
- `<`, `>`, `<=`, and `>=`

- `=` and `<>`
- `and`, `or`, and `xor`
- `implies`

You can use parentheses to group expressions, which will be evaluated from the innermost set of parentheses to the outermost.

B.3.4. Built-in Object Properties

OCL provides a set of properties on all objects in a system. You can invoke these properties in your expressions as you do any other properties. The built-in properties are:

`oclIsTypeOf (t : Type): Boolean`

Returns `true` if the tested object is exactly the same type as `t`.

`oclIsKindOf (t : Type): Boolean`

Returns `TRUE` if the tested object is the same type or a subtype of `t`.

`oclInState (s : State): Boolean`

Returns `TRUE` if the tested object is in state `s`. The states you can test must be part of a state machine attached to the classifier being tested.

`oclIsNew( ) : Boolean`

Designed to be used in a postcondition for an operation, it returns `true` if the object being tested was created as a result of executing the operation.

`oclAsType (t : Type): Type`

Returns the owning object casted to `Type`. If the object isn't a descendant of `t`, the operation is undefined.

Here are some examples of the built-in properties:

```
-- test that the instructor is an instance of Teacher
context Course
inv: self.instructor.oclIsTypeOf(Teacher)

-- cast a Date class to a java.sql.Date to verify the minutes
-- (it's very unlikely the foundationDate would be a java.sql.Date
-- so this invariant would be undefined, but this is an example
-- of using oclAsType( ))
context School
inv: self.foundationDate.oclAsType(java.sql.Date).getMinutes( ) = 0
```

B.3.5. Collections

OCL defines several types of collections that represent several instances of a classifier. The basic type is `Collection`, which is the base class for the other OCL collection classes. Quite a few operations are defined for `Collection`; see the OCL specification for the complete list.

All collections support a way to select or reject elements using the operations `select( )` and `reject( )`, respectively. To invoke an operation on a collection, you use an arrow (`->`) rather than the dot (`.`) (a dot accesses a property). The result of `select` or `reject` is another collection containing the appropriate elements. Remember that since OCL can't modify a system in any way, the original collection is unchanged. The notation for a `select` is:

```
collection->select(boolean expression)
```

So, to select students with GPAs higher than 3.0, you can use the expression:

```
context Course::getHonorsStudents( ) : Student
body: self.students->select(GPA > 3.0)
```

Or, to eliminate honor students that haven't paid their tuition:

```
context Course::getHonorsStudents( ) : Student
body: self.students->select(GPA > 3.0)->reject(tuitionPaid = false)
```

In the previous examples, the context for the `select` and `reject` statements was implied. You can explicitly name the element you want to use in the boolean expression by prefixing the expression with a label and a pipe symbol (`|`). So, a rewrite of the GPA example using a label to identify each student looks like this:

```
context Course::getHonorsStudents( ) : Student
body: self.students->select(curStudent | curStudent.GPA > 3.0)
```

Finally, you can specify the type of the element you want to evaluate. You indicate the type by placing a colon (`:`) and the classifier type after the label. Each element of the collection you are evaluating must be of the specified type, or else the expression is undefined. You can rewrite the GPA example to be even more specific and require that it evaluate only `Students`:

```
context Course::getHonorsStudents( ) : Student
body: self.students->select(curStudent : Student | curStudent.GPA > 3.0)
```

Often you will need to express a constraint across an entire collection of objects, so OCL provides the `forAll` operation that returns `true` if a given Boolean expression evaluates to `true` for all of the elements in a collection. The syntax for `forAll` is the same as that for `select` and `reject`. So, you can write a constraint that enforces that all students in a `Course` have paid their tuition:

```
context Course
inv: self.students->forAll(tuitionPaid = true)
```

As you can with `select`, you can name and type the variable used in the expression:

```
context Course
inv: self.students->forall(curStudent : Student | curStudent.tuitionPaid = tr
```

If you need to check to see if there is at least one element in a collection that satisfies a boolean expression, you can use the operation `exists`. The syntax is the same as that for `select`. The following expression ensures that at least one of the students has paid their tuition:

```
context Course
inv: self.students->exists(tuitionPaid = true)
```

Like `select`, you can name and type the variables used in the expression:

```
context Course
inv: self.students->exists(curStudent : Student | curStudent.tuitionPaid = tr
```

You can check to see if a collection is empty using the operations `isEmpty` or `notEmpty`. The following expression ensures that the school has at least one course offering:

```
context School
inv: self.Course->notEmpty( )
```